

CS 6120

Natural Language Processing

Instructor: Amir Tahmasebi, PhD

Fall 2026



Text Processing Fundamentals

Calendar

Sept 2026

SUN	MON	TUE	WED	THU	FRI	SAT
30	31	1	2	3	4	5
6	7	8	9	10	11	12
13	14	15 NLP Intro	16	17	18	19
20	21	22 Text Proc Fundamentals	23 HW1	24	25	26
27	28 HW1 due	29 Syntax and Grammar - I	30	1	2	3

Calendar

Oct 2026

SUN	MON	TUE	WED	THU	FRI	SAT
27	28	29	30	1	2	3
4	5	6 Syntax and Grammar – II LAB 1: Pytorch	7 HW2 Primer	8	9	10
11	12 HW2 due	13 Word Rep LAB 2	14 HW3	15	16	17
18	19 HW3 due	20 ML Part I	21 START: Final Project Topic Selection HW4	22	23	24
25	26 HW4 due	27 ML Part II	28	29	30	31

Calendar

Nov 2026

SUN	MON	TUE	WED	THU	FRI	SAT
1	2	3 LAB 3 LAB 4	4 DEADLINE: Final Project Topic Selection	5	6	7
8	9 One-page due	10 Language Models	11 HW5	12	13	14
15	16 HW5 due One-page due	17 NER LAB 5	18	19	20	21
22	23 One-page due	24 Q&A Machine Translation	25	26	27	28
29	30	1	2	3	4	5

Calendar

Dec 2026

SUN	MON	TUE	WED	THU	FRI	SAT
29	30	1 LLM Final Project Presentation Session 1	2	3	4	5
6	7	8 Final Project Presentation Session 2	9	10	11	12
13	14	15 BUFFER	16	17	18 DEADLINE Full Submission	19
20	21	22	23	24	25	26
27	28	29	30	31	1	2

Announcements

TA Office Hours (virtual and on zoom)

	Monday	Tuesday	Wednesday	Thursday	Friday
Zankhana	10-11am	-	-	4-5pm	-
Craig	4-5pm	-	-	-	10-11am

Announcements

Class Presentations

- *Starting next week – Presenters will be announced on Canvas*
- *2 mins per presenter*
- *[**UPLOAD one-slide on CANVAS under Class Presentation**]*

Announcements

- *Assignment 1 will be posted today*
 - **Due Date: 11:59pm, 09/28/2026**
- **Assignment submissions – not complying → 5% deduction**
 - *Submit ONLY your work material*
 - *Copy all material in one folder and **ZIP** it*
 - **File naming**
 - *Firstname.lastname.HW<number>.zip*
Eg. John.Doe.HW1.zip
- **Late Submission:**
 - *<2hrs – forgiven*
 - *2hrs-24hrs – 10 points*
 - *> 1day – 20 points*

Announcements

- *All important information will be announced on the slides as well as Canvas (under Announcements).*
- *Use Canvas discussion for questions about Assignments.*
- *For absence request, please email me prior to the date with the following information: (NOTE: make sure title of the email is “absence”)*
 - *Name*
 - *Absence Date(s)*
 - *Reason for absence:*
 - *Illness*
 - *Travel*
 - *other*

Next ...

- What is Natural Language Processing?
 - History
 - Applications
- Text Processing Fundamentals
 - Regular Expressions
 - Tokenization
 - Normalization
 - Libraries and Toolkits
- Syntax and Grammar
 - POS tagging
 - Syntax tree
- Word Representation
 - TF-IDF
 - word2vec
 - Glove
- Intro to Machine Learning
 - Logistic Regression (LR)
 - Neural Networks
 - Recurrent Neural Networks (RNN)
 - Convolutional Neural Networks (CNN)
 - Transformers
- Language Models
- NLP Use Cases
 - Named Entity Recognition
 - Machine Translation
 - QA
- Large Language Models
 - chatGPT

Text Processing Fundamentals

\n\n*****\n\nAdmission Date: [**2151-7-16**] Discharge Date: [**2151-8-4**]\n\nService:\nADDENDUM:\n\nRADIOLOGIC STUDIES: Radiologic studies also included a chest\nCT, which confirmed cavitory lesions in the left lung apex\nconsistent with infectious process/tuberculosis. This also\nmoderate-sized left pleural effusion.\n\nHEAD CT: Head CT showed no intracranial hemorrhage or mass\neffect, but old infarction consistent with past medical\nhistory.\n\nABDOMINAL CT: Abdominal CT showed lesions of\nT10 and sacrum most likely secondary to osteoporosis. These can\nbe followed by repeat imaging as an outpatient.\n\n\nFOLLOW-UP PLANS: The patient is recommended to followup with\nDr. [**First Name4 (NamePattern1) **] [**Last Name (NamePattern1) 1407**], [**Telephone/Fax (1) 1408**] within two weeks of leaving\nof the hospital.

Text Processing Fundamentals

Admission Date: 2151-7-16 Discharge Date: 2151-8-4

Service:

ADDENDUM:

RADIOLOGIC STUDIES: Radiologic studies also included a chest CT, which confirmed cavitary lesions in the left lung apex consistent with infectious process/tuberculosis. This also moderate-sized left pleural effusion.

HEAD CT: Head CT showed no intracranial hemorrhage or mass effect, but old infarction consistent with past medical history.

ABDOMINAL CT: Abdominal CT showed lesions of T10 and sacrum most likely secondary to osteoporosis. These can be followed by repeat imaging as an outpatient.

FOLLOW-UP PLANS: The patient is recommended to followup with Dr. [**First Name4 (NamePattern1) **] [**Last Name (NamePattern1) 1407**], [**Telephone/Fax (1) 1408**] within two weeks of leaving of the hospital.

Text Processing Fundamentals

Admission Date: 2151-7-16 Discharge Date: 2151-8-4

Service:

ADDENDUM:

RADIOLOGIC STUDIES: Radiologic studies also included a chest CT, which confirmed cavitary lesions in the left lung apex consistent with infectious process/tuberculosis. This also moderate-sized left pleural effusion.

HEAD CT: Head CT showed no intracranial hemorrhage or mass effect, but old infarction consistent with past medical history.

ABDOMINAL CT: Abdominal CT showed lesions of T10 and sacrum most likely secondary to osteoporosis. These can be followed by repeat imaging as an outpatient.

FOLLOW-UP PLANS: The patient is recommended to followup with Dr. [**First Name4 (NamePattern1) **] [**Last Name (NamePattern1) 1407**], [**Telephone/Fax (1) 1408**] within two weeks of leaving of the hospital.

Text Processing Fundamentals

\n\n*****\n\nAdmission Date: [**2151-7-16**] Discharge Date: [**2151-8-4**]\n\nService:\nADDENDUM:\n\nRADIOLOGIC STUDIES: Radiologic studies also included a chest\nCT, which confirmed cavitory lesions in the left lung apex\nconsistent with infectious process/tuberculosis. This also\nmoderate-sized left pleural effusion.\n\nHEAD CT: Head CT showed no intracranial hemorrhage or mass\neffect, but old infarction consistent with past medical\nhistory.\n\nABDOMINAL CT: Abdominal CT showed lesions of\nT10 and sacrum most likely secondary to osteoporosis. These can\nbe followed by repeat imaging as an outpatient.\n\n\nFOLLOW-UP PLANS: The patient is recommended to followup with\nDr. [**First Name4 (NamePattern1) **] [**Last Name (NamePattern1) 1407**], [**Telephone/Fax (1) 1408**] within two weeks of leaving\nof the hospital.

This is plain text with no markups.

The lines are artificially wrapped with new lines (whenever you see a single \n)

Text Processing Fundamentals

\n\n*****\n\nAdmission Date: [**2151-7-16**] Discharge Date: [**2151-8-4**]\n\nService:\nADDENDUM:\nRADIOLOGIC STUDIES: Radiologic studies also included a chest\nCT, which confirmed cavitory lesions in the left lung apex\nconsistent with infectious process/tuberculosis. This also\nmoderate-sized left pleural effusion.\n\nHEAD CT: Head CT showed no intracranial hemorrhage or mass\neffect, but old infarction consistent with past medical\nhistory.\n\nABDOMINAL CT: Abdominal CT showed lesions of\nT10 and sacrum most likely secondary to osteoporosis. These can\nbe followed by repeat imaging as an outpatient.\n\nFOLLOW-UP PLANS: The patient is recommended to followup with\nDr. [**First Name4 (NamePattern1) **] [**Last Name (NamePattern1) 1407**], [**Telephone/Fax (1) 1408**] within two weeks of leaving\nof the hospital.

There's punctuation like commas, apostrophes, quotes, question marks, and more.

There is even acronyms

Text Processing Fundamentals

\n\n*****\n\nAdmission Date: [**2151-7-16**] Discharge Date: [**2151-8-4**]\n\nService:\nADDENDUM:\n\nRADIOLOGIC STUDIES: Radiologic studies also included a chest\nCT, which confirmed cavitary lesions in the left lung apex\nconsistent with infectious process/tuberculosis. This also\nmoderate-sized left pleural effusion.\n\nHEAD CT: Head CT showed no intracranial hemorrhage or mass\neffect, but old infarction consistent with past medical\nhistory.\n\nABDOMINAL CT: Abdominal CT showed lesions of\nT10 and sacrum most likely secondary to osteoporosis. These can\nbe followed by repeat imaging as an outpatient.\n\n\nFOLLOW-UP PLANS: The patient is recommended to followup with\nDr. [**First Name4 (NamePattern1) **] [**Last Name (NamePattern1) 1407**], [**Telephone/Fax (1) 1408**] within two weeks of leaving\nof the hospital.

There's hyphenated descriptions like "FOLLOW-UP"

Text Processing Fundamentals

\n\n*****\n\nAdmission Date: [**2151-7-16**] Discharge Date: [**2151-8-4**]\n\nService:\nADDENDUM:\n\nRADIOLOGIC STUDIES: Radiologic studies also included a chest\nCT, which confirmed cavitary lesions in the left lung apex\nconsistent with infectious process/tuberculosis. This also\nmoderate-sized left pleural effusion.\n\nHEAD CT: Head CT showed no intracranial hemorrhage or mass\neffect, but old infarction consistent with past medical\nhistory.\n\nABDOMINAL CT: Abdominal CT showed lesions of\nT10 and sacrum most likely secondary to osteoporosis. These can\nbe followed by repeat imaging as an outpatient.\n\nFOLLOW-UP PLANS: The patient is recommended to followup with\nDr. [**First Name4 (NamePattern1) **] [**Last Name (NamePattern1) 1407**], [**Telephone/Fax (1) 1408**] within two weeks of leaving\nof the hospital.

See how the de-identified names are all replaced with the likes of 'Last Name' or 'First Name4'

Text Processing Fundamentals

\n\n*****\n\nAdmission Date: **[**2151-7-16**]** Discharge Date: **[**2151-8-4**]**\n\n\nService:\nADDENDUM:\n\nRADIOLOGIC STUDIES: Radiologic studies also included a chest\nCT, which confirmed cavitary lesions in the left lung apex\nconsistent with infectious process/tuberculosis. This also\nmoderate-sized left pleural effusion.\n\nHEAD CT: Head CT showed no intracranial hemorrhage or mass\neffect, but old infarction consistent with past medical\nhistory.\n\nABDOMINAL CT: Abdominal CT showed lesions of\nT10 and sacrum most likely secondary to osteoporosis. These can\nbe followed by repeat imaging as an outpatient.\n\n\nFOLLOW-UP PLANS: The patient is recommended to followup with\nDr. **[**First Name4 (NamePattern1) **]** **[**Last Name (NamePattern1) 1407**]**, **[**Telephone/Fax (1) 1408**]** within two weeks of leaving\nof the hospital.

Notice how dates might be required to handle (if they are all not in the same format)

Text Processing Fundamentals

\n\n*****\n\nAdmission Date: [**2151-7-16**] Discharge Date: [**2151-8-4**]\n\n\nService:\nADDENDUM:\n\nRADIOLOGIC STUDIES: Radiologic studies also included a chest\nCT, which confirmed cavitary lesions in the left lung apex\nconsistent with infectious process/tuberculosis. This also\nmoderate-sized left pleural effusion.\n\nHEAD CT: Head CT showed no intracranial hemorrhage or mass\neffect, but old infarction consistent with past medical\nhistory.\n\nABDOMINAL CT: Abdominal CT showed lesions of\nT10 and sacrum most likely secondary to osteoporosis. These can\nbe followed by repeat imaging as an outpatient.\n\n\nFOLLOW-UP PLANS: The patient is recommended to followup with\nDr. [**First Name4 (NamePattern1) **] [**Last Name (NamePattern1) 1407**], [**Telephone/Fax (1) 1408**] within two weeks of leaving\nof the hospital.

Notice some formatting: such as :
\n\n*****\n\n

Text Processing Fundamentals

- Regular Expressions
- Tokenization
- Normalization
- Libraries and Toolkits

Regular Expressions

A SIMPLE SEARCH

Suppose you are tasked to find every record in a clinical database with the mention a specific medication “. Misspelled medication name is a common challenge in such manually entered data.

Example 1: Amoxicillin

Misspellings might include "Amoxycillin", "Amoxycilin", "Amoxicilin", "Amoxcilin"

Example 2: Metformin

Misspellings might include "Metphormin", "Metformine", "Metforimn"

Regular Expressions

VALIDATING THE FORMAT OF A FREE TEXT FIELD

Suppose you are tasked to verify the format in a corpus of clinical records created using free-text fields to capture medication product name and corresponding dose.

<medication name> , <dose> <unit>

Examples:

Hydro-Chlorothiazide	25.5	
Ziagen	200	mg
Zerit		mg
Insulin	20	cc
Dapsone	100	g
Kaletra	3	tabs

Regular Expressions

COMPLEX SEARCH PATTERN

You have a dataset that contains information related to serious Adverse Drug Events (ADE). You are tasked to identify any clinical records that mentions two specific ADEs.

Examples:

Headache and nausea

Nausae and headache

Patient reported headache and nausea

Pt. Rptd. Head ache and nausea

Naus. And hdache reported

Pt reported headache at admission; patient later reported nausea.

Regular Expressions

EXTRACTING TEXT

You have a dataset that contains information related to serious Adverse Drug Events (ADE) reported by patients themselves. You are tasked to extract ADEs in any clinical records that reported BY PATIENTS and NOT Physicians.

Examples:

Patient reported headache and nausea.

MD noticed rash.

Pt. Rptd. Backache.

patient reported seeing spots.

Elevated pulse and labored breathing.

Headache.

Regular Expressions

TEXT NORMALIZATION

You are tasked to standardize various spellings and abbreviations for “Multi-vitamin”.

Examples:

multivitamin

multi-vitamin

multi-vita

multivit

multi-vit

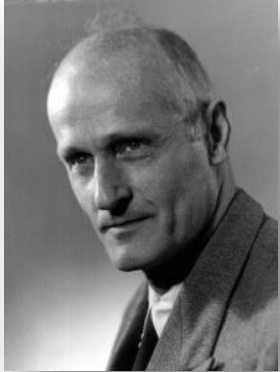
multi vitamin

Regular Expressions

Definition:

A set of "instruction" that is given to a function on what and how to search, match, or replace a set of strings.

History:



Stephen Cole Kleene

In 1951, mathematician Stephen Cole Kleene described the concept of a regular language, a language that is recognizable by a finite automaton and formally expressible using regular expressions.

In the mid-1960s, computer science pioneer Ken Thompson, one of the original designers of Unix, implemented pattern matching in the QED text editor using Kleene's notation.



Ken Thompson

Regular Expressions

Use Cases:

Data pre-processing

Rule-based information mining systems

Pattern matching

Text feature engineering

Web scraping

Data extraction

...

Regular Expressions

Use Cases:

Data pre-processing

Rule-based information mining systems

Pattern matching

Text feature engineering

Web scraping

Data extraction

...

Python, Java, and Perl all support regex functionality, as do most Unix tools and many text editors.

Regular Expressions: *in Python*

In Python a regular expression search is typically written as:

```
import re
match = re.search(pat, str)
```

```
import re

str = 'an example word:cat!!'
match = re.search(r'word:\w\w\w', str)
# If-statement after search() tests if it succeeded
if match:
    print('found', match.group()) ## 'found word:cat'
else:
    print('did not find')
```

Regular Expressions: *Basics*

- a, X, 9, <: ordinary characters just match themselves exactly.

NOTE: The meta-characters do not match themselves because they have special meanings: . ^ \$ * + ? { [] \ | ()

```
In [ ]:
```

```
## Search for pattern 'iii' in string 'piiig'.
```

```
## All of the pattern must match, but it may appear anywhere.
```

```
## On success, match.group() is matched text.
```

```
match = re.search(r'iii', 'piiig')
```

```
test(match)
```


Regular Expressions: *Basics*

- a, X, 9, <: ordinary characters just match themselves exactly.

NOTE: The meta-characters do not match themselves because they have special meanings: . ^ \$ * + ? { [] \ | ()

```
In [ ]:
```

```
## Search for pattern 'iii' in string 'piiig'.
```

```
## All of the pattern must match, but it may appear anywhere.
```

```
## On success, match.group() is matched text.
```

```
match = re.search(r'iii', 'piiig')
```

```
test(match)
```

Regular Expressions: *Basics*

Note: Raw String

By using a raw string (denoted by the prefix `r`), you tell Python to treat backslashes as literal characters and not as escape characters. This makes writing regular expressions more straightforward and readable.

e.g., a word boundary in a regular expression can be written as `r'\b'` in a raw string instead of `'\\b'` in a regular string.

Regular Expressions: *Basics*

- . (a period): matches any single character except newline '\n'

In []:

. = any char but \n

```
match = re.search(r'..g', 'piiig')  
test(match)
```

Regular Expressions: *Basics*

- `\d` (lowercase d): matches a decimal digit char 0...9
- `\w` (lowercase w): matches a "word" character: a letter or digit or underbar [a-zA-Z0-9_].

NOTE: although "word" is the mnemonic for this, it only matches a single word char, not a whole word. `\W` (upper case W) matches any non-word character.

```
In [ ]:  
## \d = digit char,
```

```
match = re.search(r'\d\d\d', 'p123g')  
test(match)
```

```
In [ ]:  
## \w = word char
```

```
match = re.search(r'\w\w\w', '@@abcd!!')  
test(match)
```

Regular Expressions: *Basics*

- `\b`: boundary between word and non-word
- `\s` (lowercase s): matches a single whitespace character (space, newline, return, tab, form [`\n\r\t\f`]).
- `\S` (upper case S): matches any non-whitespace character.

Regular Expressions: *Basics*

- `\t`, `\n`, `\r`: tab, newline, return
- `^` = start: match the start of the string
- `$` = end: match the end of the string
- `\`: inhibit the "specialness" of a character.
 - Example,
 - use `\.` to match a period
 - `\\` to match a slash.

NOTE: If you are unsure if a character has special meaning, such as '@', you can try putting a slash in front of it, `\@`. If its not a valid escape sequence, like `\c`, your python program will halt with an error.

Regular Expressions: *Basics*

Repetitions

+ : 1 or more occurrences of the pattern to its left, e.g. 'i+' = one or more i's

***** : 0 or more occurrences of the pattern to its left

? : match 0 or 1 occurrences of the pattern to its left

Leftmost & Largest

First the search finds the leftmost match for the pattern, and **second** it tries to use up as much of the string as possible

Regular Expressions: *Basics*

Repetitions: + → 1 or more

```
In [ ]:
```

```
## i+ = one or more i's, as many as possible.
```

```
match = re.search(r'pi+', 'piiiig') # => found, match.group() == "piii"  
test(match)
```

```
## In this example, note that it does not get to the second set of i's.
```

```
match = re.search(r'i+', 'piigiinii') # => found, match.group() == "ii"  
test(match)
```


Regular Expressions: *Basics*

Repetitions: * → 0 or more

```
## \s* = zero or more whitespace chars
## Here look for 3 digits, possibly separated by whitespace.
```

```
match = re.search(r'\d\s*\d\s*\d', 'xx1 2 3xx')
test(match)
```

```
match = re.search(r'\d\s*\d\s*\d', 'xx12 3xx')
test(match)
```

```
match = re.search(r'\d\s*\d\s*\d', 'xx123xx')
test(match)
```

Regular Expressions: *Basics*

Repetitions:

$a?$ $\rightarrow$ 0 or 1 match

$a\{n\}$ $\rightarrow$ exactly n match

$a\{n,\}$ $\rightarrow$ n or more match

$a\{n,m\}$ $\rightarrow$ between n and m matches

```
test_phrase = 'sdsd...sssddd...sdddsddd...dsds...dsssss...sdddd'
```

```
test_patterns = [ 'sd*',      # s followed by zero or more d's
                  'sd+',      # s followed by one or more d's
                  'sd?',      # s followed by zero or one d's
                  'sd{3}',     # s followed by three d's
                  'sd{2,3}',   # s followed by two to three d's
                  ]
```

```
multi_re_find(test_patterns, test_phrase)
```

Regular Expressions: *Basics*

Square Brackets:

used to indicate a set of chars, so `[abc]` matches 'a' or 'b' or 'c'.

NOTE: The codes `\w`, `\s` etc. work inside square brackets too with the one exception that dot (.) just means a literal dot.

Regular Expressions: *Basics*

Negation

‘^’ inside brackets (immediately after the opening square bracket) indicates negation.

- `[^0-9]`: Matches any character that is not a digit.
- `[^A-Za-z]`: Matches any character that is not a letter.
- `[^aeiou]`: Matches any character that is not a vowel.

Regular Expressions

'[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}'

Regular Expressions

```
'[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}'
```

Email matcher

Name of sender

1. Has alphabets and numbers and dots
2. Occurs before @

Domain name:

1. Has alphabets
2. No dots
3. Occurs after @ and before a dot

Jane.doe1971@gmail.com

@ symbol

TLD

1. Occurs after the dot and contains alphabets only

Regular Expressions

' **[a-zA-Z0-9._%+-]** + @ [a-zA-Z0-9.-] + \. [a-zA-Z] {2,} '

[a-zA-Z0-9._%+-] is a character class that matches:

- a-z: Any lowercase letter.
- A-Z: Any uppercase letter.
- 0-9: Any digit.
- .: A dot.
- _: An underscore.
- %: A percent sign.
- +: A plus sign.
- -: A hyphen.

Regular Expressions

'[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}'

'+':

The + after the character class means "one or more of the preceding element." So, this part of the pattern matches one or more of any of the characters listed in the class.

Regular Expressions

```
'[a-zA-Z0-9._%+-]+@'[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}'
```

'@':

This is a literal character match for "@" – the symbol used in all email addresses to separate the local part from the domain part.

Regular Expressions

```
'[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}'
```

'[a-zA-Z0-9.-]+':

Another character class, similar to the first part, but this time for the domain part of the email.

This class includes lowercase and uppercase letters, digits, dots, and hyphens.

Again, the + means "one or more of the preceding element."

Regular Expressions

```
'[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}'
```

`\.`:

This is a literal character match for a dot. In regex, a dot is a special character that matches almost any character, so it needs to be escaped with a backslash (`\`) to match an actual dot.

Regular Expressions

```
'[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}'
```

'[a-zA-Z]{2,}':

This final part matches the top-level domain (like .com, .org, .net, etc.).

The {2,} quantifier specifies that this part of the pattern must be at least two characters long, which is a common minimum length for top-level domains.

Regular Expressions: *Basics*

Group Extraction:

allows you to pick out parts of the matching text by adding parenthesis ()

() helps establish logical "groups" inside of the match text.

On a successful search, `match.group(1)` is the match text corresponding to the 1st left parenthesis, and `match.group(2)` is the text corresponding to the 2nd left parenthesis.

The plain `match.group()` is still the whole match text as usual.

Regular Expressions: *Basics*

Group Extraction:

```
In [ ]:
str = 'purple alice-b@google.com monkey dishwasher'
match = re.search('([\w.-]+)@([\w.-]+)', str)

if match:
    print(match.group()) ## 'alice-b@google.com' (the whole match)
    print(match.group(1)) ## 'alice-b' (the username, group 1)
    print(match.group(2)) ## 'google.com' (the host, group 2)
```

Regular Expressions: *Basics*

findall

matches and returns them as a list of strings, with each string representing one match.

```
In [ ]:
## Suppose we have a text with many email addresses

str = 'purple alice@google.com, blah monkey bob@abc.com blah dishwasher'

## Here re.findall() returns a list of all the found email strings
emails = re.findall(r'[\w\.-]+@[\w\.-]+', str)

for email in emails:
    print(email)
```

Regular Expressions: *Basics*

Greedy vs Non-greedy

Suppose you have text with tags in it: `<b>foo</b>` and `<i>so on</i>`

Goal: match each tag with the pattern '`<.*>`'

GREEDY `(<.*>)`

NON-GREEDY `(<.*?>)`

Regular Expressions: *Basics*

Substitute

Searches for all the instances of pattern in the given string, and replaces them

The replacement string can include '\1', '\2' which refer to the text from group(1), group(2), and so on from the original matching text.

```
In [ ]:  
str = 'purple alice@google.com, blah monkey bob@abc.com blah dishwasher'  
  
re.sub(r'([\w\.-]+)@([\w\.-]+)', r'\1@yo-yo-dyne.com', str)
```

Regular Expressions: *Basics*

Compile

Compiles a regular expression pattern into a regular expression object.

This object can then be used to perform various operations like matching, searching, finding all occurrences, and replacing.

```
import re

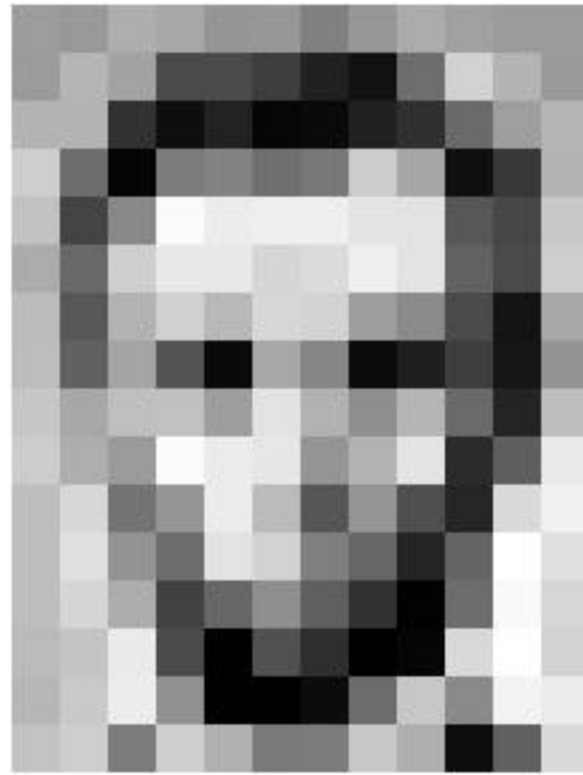
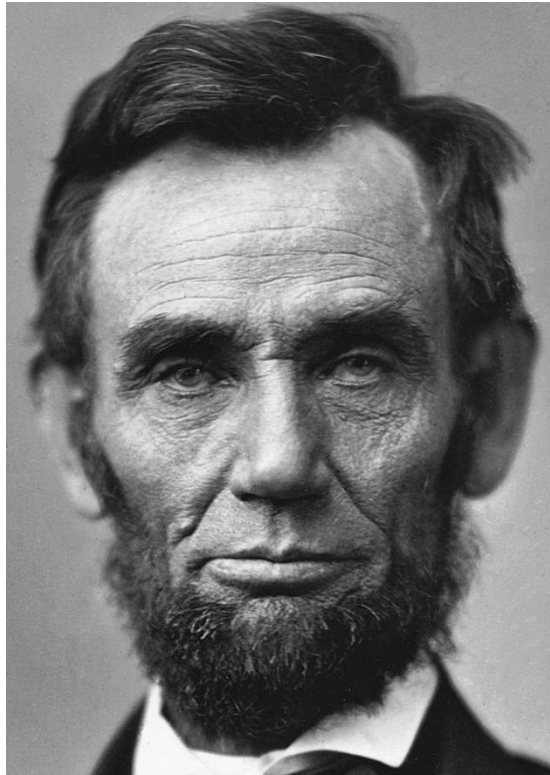
# Compile the pattern
pattern = re.compile(r'\d+') # Matches one or more digits

# Using the pattern to search in a string
result = pattern.search("The answer is 42")
if result:
    print("Found number:", result.group())
```

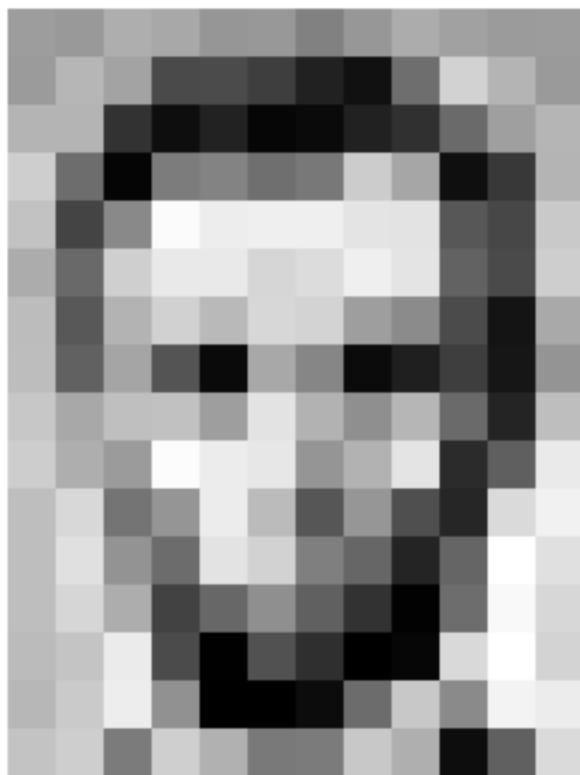
Text Processing Fundamentals

- Regular Expressions
- **Tokenization**
- Normalization
- Libraries and Toolkits

Tokenization



Tokenization



157	153	174	168	150	152	129	151	172	161	155	156
155	182	163	74	75	62	33	17	110	210	180	154
180	180	50	14	34	6	10	33	48	106	159	181
206	109	5	124	131	111	120	204	166	15	56	180
194	68	137	251	237	239	239	228	227	87	71	201
172	106	207	233	233	214	220	239	228	98	74	206
188	88	179	209	185	215	211	158	139	75	20	169
189	97	165	84	10	168	134	11	31	62	22	148
199	168	191	193	158	227	178	143	182	106	36	190
205	174	155	252	236	231	149	178	228	43	95	234
190	216	116	149	236	187	86	150	79	38	218	241
190	224	147	108	227	210	127	102	36	101	255	224
190	214	173	66	103	143	96	50	2	109	249	215
187	196	235	75	1	81	47	0	6	217	255	211
183	202	237	145	0	0	12	108	200	138	243	236
195	206	123	207	177	121	123	200	175	13	96	218

157	153	174	168	150	152	129	151	172	161	155	156
155	182	163	74	75	62	33	17	110	210	180	154
180	180	50	14	34	6	10	33	48	106	159	181
206	109	5	124	131	111	120	204	166	15	56	180
194	68	137	251	237	239	239	228	227	87	71	201
172	106	207	233	233	214	220	239	228	98	74	206
188	88	179	209	185	215	211	158	139	75	20	169
189	97	165	84	10	168	134	11	31	62	22	148
199	168	191	193	158	227	178	143	182	106	36	190
205	174	155	252	236	231	149	178	228	43	95	234
190	216	116	149	236	187	86	150	79	38	218	241
190	224	147	108	227	210	127	102	36	101	255	224
190	214	173	66	103	143	96	50	2	109	249	215
187	196	235	75	1	81	47	0	6	217	255	211
183	202	237	145	0	0	12	108	200	138	243	236
195	206	123	207	177	121	123	200	175	13	96	218

Tokenization



Tokenization

"This is a input text."

Tokenization



[CLS]	This	is	a	input	.	[SEP]
101	2023	2003	1037	7953	1012	102

Embeddings



0.0390,	-0.0558,	-0.0440,	0.0119,	0069,	0.0199,	-0.0788,
-0.0123,	0.0151,	-0.0236,	-0.0037,	0.0057,	-0.0095,	0.0202,
-0.0208,	0.0031,	-0.0283,	-0.0402,	-0.0016,	-0.0099,	-0.0352,
...	...	...	...	...	...	...

Tokenization

Tokenization is the process of splitting a piece of text into smaller units, called tokens.

Tokens can be words, numbers, or punctuation marks.

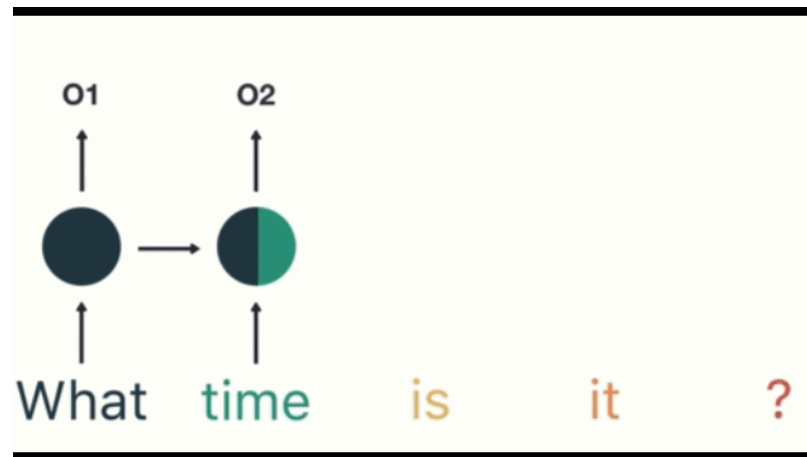
tokenization can be broadly classified into 3 types:

- word,
- character,
- subword (n-gram characters)



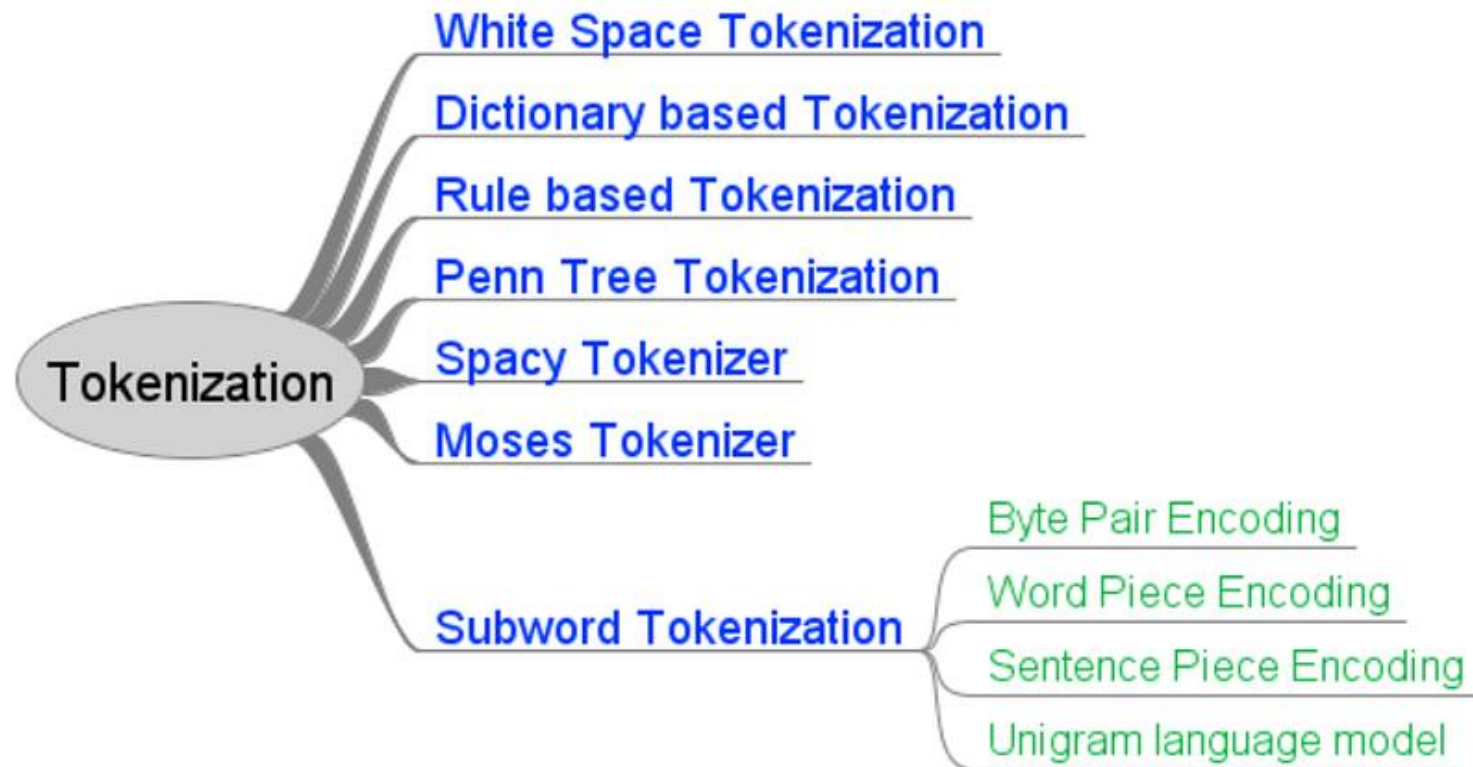
Tokenization: *WHY?*

- Building blocks of text in the natural language.
 - Most of the preprocessing and modeling happens at the token level.
 - Neural Network architectures process individual tokens to make sense of the document.



<https://towardsdatascience.com/illustrated-guide-to-recurrent-neural-networks-79e5eb8049c9>

Tokenization: *Techniques*



Tokenization: *word-level*

- Simplest tokenization methods
- breaking text into individual words using a *delimiter*
- ***Delimiter:*** white-space, punctuation, etc.



“This is a lame example, isn’t it?”

Tokenization: *word-level*

- Simplest tokenization methods
- breaking text into individual words using a *delimiter*
- ***Delimiter***: white-space, punctuation, etc.



“This is a lame example, isn’t it?”

[“This”, “is”, “a”, “lame”, “example,”, “is”, “n’t”, “it?”]

[“This”, “is”, “a”, “lame”, “example,”, “isn’t”, “it?”]

[“This”, “is”, “a”, “lame”, “example,”, “isn’t”, “it”, “?”]

Tokenization: *word-level*

- Simplest tokenization methods
- breaking text into individual words using a *delimiter*
- ***Delimiter***: white-space, punctuation, etc.



“This is a lame example, isn’t it?”

[“This”, “is”, “a”, “lame”, “example,”, “is”, “n’t”, “it?”]

white-space

[“This”, “is”, “a”, “lame”, “example,”, “isn’t”, “it?”]

[“This”, “is”, “a”, “lame”, “example,”, “isn’t”, “it”, “?”]

Tokenization: *word-level*

- Simplest tokenization methods
- breaking text into individual words using a *delimiter*
- ***Delimiter:*** white-space, punctuation, etc.



white-space +
punctuation

“This is a lame example, isn’t it?”

[“This”, “is”, “a”, “lame”, “example,”, “isn’t”, “it?”]

[“This”, “is”, “a”, “lame”, “example”, “,”, “isn’t”, “it”, “?”]

[“This”, “is”, “a”, “lame”, “example”, “,”, “isn”, “’”, “t”, “it”, “?”]

Tokenization: *word-level*

- Simplest tokenization methods
- breaking text into individual words using a *delimiter*
- ***Delimiter***: white-space, punctuation, etc.



white-space +
punctuation

“This is a lame example, isn’t it?”

[“This”, “is”, “a”, “lame”, “example,”, “isn’t”, “it?”]

[“This”, “is”, “a”, “lame”, “example”, “,”, “isn’t”, “it”, “?”]

[“This”, “is”, “a”, “lame”, “example”, “,”, “isn”, “’”, “t”, “it”, “?”]

Tokenization: *word-level*

- Simplest tokenization methods
- breaking text into individual words using a *delimiter*
- ***Delimiter***: white-space, punctuation, etc.



“This is a lame example, isn’t it?”

white-space

[“This”, “is”, “a”, “lame”, “example,”, “isn’t”, “it?”]

white-space +
punctuation

[“This”, “is”, “a”, “lame”, “example”, “,”, “isn”, “’”, “t”, “it”, “?”]

Tokenization: *word-level*

- What if there are plural forms?



“These are lame **examples**, aren’t they?”

OOV*

Tokenization: *word-level*

- What if there are plural forms?
- What if there are misspellings?



“These are lame **examples**, aren’t they?”

“These are lame **exmples**, aren’t they?”

OOV*

Tokenization: *word-level*

Limitations:

- very large vocabulary size,
- large number of OOV tokens,
- different meaning of very similar words.



NOTE: Transformer XL, uses space and punctuation tokenization and has a vocabulary size of 267,735.

Tokenization: *character-level*

- Split the raw text into individual characters.
- Simple and can greatly reduce memory and time complexity.



“This is a lame example, isn’t it?”

[“T”, “h”, “i”, “s”, “i”, “s”, “a”, ...]

Tokenization: *character-level*

Limitations:

- very long sequences
- less meaningful individual tokens.



Tokenization: *subword-level*



- A solution between word and character level tokenizer.
 - *Address issues such as vocabulary explosion and out-of-vocabulary (OOV) problems.*
 - *Better handling of rare and morphologically rich words.*

Tokenization: *subword-level*

- **Principles:**

- *Do not split the frequently used words into smaller subwords.*
- *Split the rare words into smaller meaningful subwords.*



- **Benefits:**

- Helps the model learn that the words with the same root word are similar in meaning.
- Help the model learn that “tokenization” and “modernization” are made up of different root words but have the same suffix “ization” and are used in the same syntactic situations.

Tokenization: *subword-level*

- The subword-based tokenization algorithms generally use a special symbol to indicate which word is the start of the token and which word is the completion of the start of the token.
 - example, “tokenization”: “token” and “##ization”

Tokenization: *subword-level*

Common algorithms:

- Byte-Pair Encoding (BPE) (Sennrich et al., 2016)
- Unigram language modeling tokenization (Kudo, 2018)
- WordPiece (Schuster and Nakajima, 2012)
- SentencePiece (Kudo and Richardson, 2018)

Tokenization: *subword-level*

Common algorithms:

- Byte-Pair Encoding (BPE) (Sennrich et al., 2016)
- Unigram language modeling tokenization (Kudo, 2018)
- WordPiece (Schuster and Nakajima, 2012)
- SentencePiece (Kudo and Richardson, 2018)

Methodology:

- A token learner that takes a raw training corpus and induces a vocabulary (a set of tokens).
- A token segmenter that takes a raw test sentence and tokenizes it according to that vocabulary

Tokenization: *subword-level*

Common algorithms:

- **Byte-Pair Encoding (BPE) (Sennrich et al., 2016)**
- Unigram language modeling tokenization (Kudo, 2018)
- WordPiece (Schuster and Nakajima, 2012)
- SentencePiece (Kudo and Richardson, 2018)

Tokenization: *subword-level*

Byte-Pair Encoding (BPE) (Sennrich et al., 2016)

- Simple form of data compression algorithm in which the most common pair of consecutive bytes of data is replaced with a byte that does not occur in that data.
- It was first described in the article “A New Algorithm for Data Compression” published in 1994.
- BPE is used in language models like GPT-2, RoBERTa, XLM, FlauBERT, etc.

Tokenization: *subword-level*

Byte-Pair Encoding (BPE)

function BYTE-PAIR ENCODING(strings C , number of merges k) **returns** vocab V

$V \leftarrow$ all unique characters in C # initial set of tokens is characters

for $i = 1$ to k do # merge tokens til k times

$t_L, t_R \leftarrow$ Most frequent pair of adjacent tokens in C

$t_{NEW} \leftarrow t_L + t_R$ # make new token by concatenating

$V \leftarrow V + t_{NEW}$ # update the vocabulary

 Replace each occurrence of t_L, t_R in C with t_{NEW} # and update the corpus

return V

Tokenization: *subword-level*

Byte-Pair Encoding (BPE)

function BYTE-PAIR ENCODING(strings C , number of merges k) **returns** vocab V

$V \leftarrow$ all unique characters in C # initial set of tokens is characters

for $i = 1$ **to** k **do** # merge tokens til k times

$t_L, t_R \leftarrow$ Most frequent pair of adjacent tokens in C

$t_{NEW} \leftarrow t_L + t_R$ # make new token by concatenating

$V \leftarrow V + t_{NEW}$ # update the vocabulary

 Replace each occurrence of t_L, t_R in C with t_{NEW} # and update the corpus

return V

Tokenization: *subword-level*

Byte-Pair Encoding (BPE)

Example: aaabdaaabc

Tokenization: *subword-level*

Byte-Pair Encoding (BPE)

Example: aabdaabac

Tokenization: *subword-level*

Byte-Pair Encoding (BPE)

Example: aaabdaaabc

$Z = aa \rightarrow ZabdZabc$

Tokenization: *subword-level*

Byte-Pair Encoding (BPE)

Example: aaabdaaabc

ZabdZabc

Tokenization: *subword-level*

Byte-Pair Encoding (BPE)

Example: aaabdaaabad

$Y = ab \rightarrow ZYdZYac$

Tokenization: *subword-level*

Byte-Pair Encoding (BPE)

Example: aaabdaaabc

ZYdZYac

Tokenization: *subword-level*

Byte-Pair Encoding (BPE)

Example: aaabdaaabad

$X = ZY \rightarrow \mathbf{XdXac}$

Tokenization: *subword-level*

Byte-Pair Encoding (BPE)

Example: aaabdaaabac

Z = aa → ZabdZabac

Y = ab → ZYdZYac

X = ZY → XdXac

Tokenization: *subword-level*

Byte-Pair Encoding (BPE)

Example: aaabdaaabc

ZabdZabac

$Y = ab \rightarrow \dots \rightarrow XdXac$

What if... $Y = Za$

ZabdZabac $\rightarrow$ YbdYbac, then next merge Yb $\rightarrow$ X gives XdXac

Tokenization: *subword-level*

Byte-Pair Encoding (BPE)

Example:

Corpus: old, older, highest, and lowest

{“old”: 7, “older”: 3, “highest”: 9, “lowest”: 4}

Tokenization: *subword-level*

Byte-Pair Encoding (BPE)

Example:

Corpus: old, older, highest, and lowest

1. add a special end token “</w>” at the end of each word.

{“old</w>”: 7, “older</w>”: 3, “highest</w>”: 9, “lowest</w>”: 4}

Tokenization: *subword-level*

Byte-Pair Encoding (BPE)

Example:

Corpus: old, older, highest, and lowest

Number	Token	Frequency
1	</w>	23
2	o	14
3	l	14
4	d	10
5	e	16
6	r	3
7	f	9
8	i	9
9	n	9
10	s	13
11	t	13
12	w	4

Tokenization: *subword-level*

Byte-Pair Encoding (BPE)

Example:

Corpus: old, older, highest, and lowest

Iteration 1:

Most frequent: 'e'

Number	Token	Frequency
1	</w>	23
2	o	14
3	l	14
4	d	10
5	e	16
6	r	3
7	f	9
8	i	9
9	n	9
10	s	13
11	t	13
12	w	4

Tokenization: *subword-level*

Byte-Pair Encoding (BPE)

Example:

Corpus: old, older, highest, and lowest

Iteration 1:

'e' , 's' $\rightarrow$ 'es'

Number	Token	Frequency
1	</w>	23
2	o	14
3	l	14
4	d	10
5	e	$16 - 13 = 3$
6	r	3
7	f	9
8	i	9
9	n	9
10	s	$13 - 13 = 0$
11	t	13
12	w	4
13	es	$9 + 4 = 13$

Tokenization: *subword-level*

Byte-Pair Encoding (BPE)

Example:

Corpus: old, older, highest, and lowest

Iteration 2:

'es' , 't' $\rightarrow$ 'est'

Number	Token	Frequency
1	</w>	23
2	o	14
3	l	14
4	d	10
5	e	$16 - 13 = 3$
6	r	3
7	f	9
8	i	9
9	n	9
10	s	$13 - 13 = 0$
11	t	$13 - 13 = 0$
12	w	4
13	es	$9 + 4 = 13 - 13 = 0$
14	est	13

Tokenization: *subword-level*

Byte-Pair Encoding (BPE)

Example:

Corpus: old, older, highest, and lowest

Iteration 3:

'est' , '</w>' $\rightarrow$ 'est/w>'

Number	Token	Frequency
1	</w>	$23 - 13 = 10$
2	o	14
3	l	14
4	d	10
5	e	$16 - 13 = 3$
6	r	3
7	f	9
8	i	9
9	n	9
10	s	$13 - 13 = 0$
11	t	$13 - 13 = 0$
12	w	4
13	es	$9 + 4 = 13 - 13 = 0$
14	est	$13 - 13 = 0$
15	est</w>	13

Tokenization: *subword-level*

Byte-Pair Encoding (BPE)

Example:

Corpus: old, older, highest, and lowest

Iteration 4:

'o' , 'l' $\rightarrow$ 'ol'

Number	Token	Frequency
1	</w>	23
2	o	$14 - 10 = 4$
3	l	$14 - 10 = 4$
4	d	10
5	e	$16 - 13 = 3$
6	r	3
7	f	9
8	i	9
9	n	9
10	s	$13 - 13 = 0$
11	t	$13 - 13 = 0$
12	w	4
13	es	$9 + 4 = 13 - 13 = 0$
14	est	13
15	ol	$7 + 3 = 10$

Tokenization: *subword-level*

Byte-Pair Encoding (BPE)

Example:

Corpus: old, older, highest, and lowest

Iteration 5:

'ol' , 'd' $\rightarrow$ 'old'

Number	Token	Frequency
1	</w>	$23 - 13 = 10$
2	o	$14 - 10 = 4$
3	l	$14 - 10 = 4$
4	d	$10 - 10 = 0$
5	e	$16 - 13 = 3$
6	r	3
7	f	9
8	i	9
9	n	9
10	s	$13 - 13 = 0$
11	t	$13 - 13 = 0$
12	w	4
13	es	$9 + 4 = 13 - 13 = 0$
14	est	$13 - 13 = 0$
15	est</w>	13
16	ol	$7 + 3 = 10 - 10 = 0$
17	old	$7 + 3 = 10$

Tokenization: *subword-level*

Byte-Pair Encoding (BPE)

Example:

Corpus: old, older, highest, and lowest

FINAL

Number	Token	Frequency
1	</w>	10
2	o	4
3	l	4
4	e	3
5	r	3
6	f	9
7	i	9
8	n	9
9	w	4
10	est</w>	13
11	old	10

Tokenization: *subword-level*

Byte-Pair Encoding (BPE)

Example:

["the</w>", "high", "est</w>", "range</w>", "in</w>", "Seattle</w>"]

DECODE (?)

["the", "highest", "range", "in", "Seattle"]

["the", "high", "estrangle", "in", "Seattle"]

Number	Token	Frequency
1	</w>	10
2	o	4
3	l	4
4	e	3
5	r	3
6	f	9
7	i	9
8	n	9
9	w	4
10	est</w>	13
11	old	10

Tokenization: *subword-level*

Byte-Pair Encoding (BPE)

Example:

["the</w>", "high", "est</w>", "range</w>", "in</w>", "Seattle</w>"]

DECODE (?)

["the", "highest", "range", "in", "Seattle"]

["the", "high", "estranger", "in", "Seattle"]

Number	Token	Frequency
1	</w>	10
2	o	4
3	l	4
4	e	3
5	r	3
6	f	9
7	i	9
8	n	9
9	w	4
10	est</w>	13
11	old	10

Tokenization: *subword-level*

Byte-Pair Encoding (BPE)

Note

- Whitespace delimited languages need to be pre-tokenized before being fed to BPE.
- As a result: BPE tokenization does not cross word boundaries.

e.g., a subword "e_n" won't be generated from "The nodule."

Tokenization: *subword-level*

Common algorithms:

- Byte-Pair Encoding (BPE) (Sennrich et al., 2016)
- **Unigram language modeling tokenization (Kudo, 2018)**
- WordPiece (Schuster and Nakajima, 2012)
- SentencePiece (Kudo and Richardson, 2018)

Tokenization: *subword-level*

Unigram language modeling tokenization (Kudo, 2018)

- In contrast to BPE, initializes its base vocabulary to a large number of symbols and progressively trims down each symbol to obtain a smaller vocabulary that is a desired size.
- Unigram tokenization treats tokenization as a **probabilistic process**. Instead of merging frequent character pairs, it starts with a large pool of candidate tokens and learns which tokens best explain the training corpus using a probabilistic model.
- Unigram is not used directly for any of the models in the transformers, but it's used in conjunction with SentencePiece.

Tokenization: *subword-level*

Unigram language modeling tokenization (Kudo, 2018)

Background:

Language Model

Given a sequence of words, predict what word can come next.

Unigram Language Model

Prediction assume independence between consecutive tokens

$$P(x_N) = p(x_1) \times p(x_2) \times \cdots \times p(x_{N-1}) = \prod_{i=1}^{N-1} p(x_i)$$

Tokenization: *subword-level*

Unigram language modeling tokenization (Kudo, 2018)

METHOD:

- defines a loss (often defined as the log-likelihood) over the training data given the current vocabulary.
- for each symbol in the vocabulary, computes how much the overall loss would increase if the symbol was to be removed from the vocabulary.
- removes $p\%$ (with p usually being 10% - 20%) of the symbols whose loss increase is the lowest,
- repeat until the vocabulary has reached the desired size.

Tokenization: *subword-level*

Unigram language modeling tokenization (Kudo, 2018)

Corpus

token	Freq.
hug	10
pug	12
lug	5
bug	4
dug	5

Tokenization: *subword-level*

Unigram language modeling tokenization (Kudo, 2018)

Corpus

token	Freq.
hug	10
pug	12
lug	5
bug	4
dug	5

Vocabulary

h	hu
u	ug
g	pu
l	lu
b	du
d	bu
p	

Tokenization: *subword-level*

Unigram language modeling tokenization (Kudo, 2018)

Vocabulary

h	hu
u	ug
g	pu
l	lu
b	du
d	bu
p	

Iteration 1

Step 1

h	10/180	hu	10/180
u	36/180	ug	36/180
g	36/180	pu	12/180
l	5/180	lu	5/180
b	4/180	du	5/180
d	5/180	bu	4/180
p	12/180		

Tokenization: *subword-level*

Unigram language modeling tokenization (Kudo, 2018)

Vocabulary

h	hu
u	ug
g	pu
l	lu
b	du
d	bu
p	

Iteration 1

Step 1

h	10/180	hu	10/180
u	36/180	ug	36/180
g	36/180	pu	12/180
l	5/180	lu	5/180
b	4/180	du	5/180
d	5/180	bu	4/180
p	12/180		

hug

$h + u + g =$

$hu + g =$

$h + ug =$

$hug =$

Tokenization: *subword-level*

Unigram language modeling tokenization (Kudo, 2018)

Vocabulary

h	hu
u	ug
g	pu
l	lu
b	du
d	bu
p	

Iteration 1

Step 1

h	10/180	hu	10/180
u	36/180	ug	36/180
g	36/180	pu	12/180
l	5/180	lu	5/180
b	4/180	du	5/180
d	5/180	bu	4/180
p	12/180		

hug

$$h + u + g = \frac{10}{180} \times \frac{36}{180} \times \frac{36}{180} = 2.22e^{-03}$$

$$hu + g = \frac{10}{180} \times \frac{36}{180} = 1.11e^{-02}$$

$$h + ug = \frac{10}{180} \times \frac{36}{180} = 1.11e^{-02}$$

$$hug = 0$$

Tokenization: *subword-level*

Unigram language modeling tokenization (Kudo, 2018)

Vocabulary

h	hu
u	ug
g	pu
l	lu
b	du
d	bu
p	

Iteration 1

Step 1

h	10/180	hu	10/180
u	36/180	ug	36/180
g	36/180	pu	12/180
l	5/180	lu	5/180
b	4/180	du	5/180
d	5/180	bu	4/180
p	12/180		

hug

$$h + u + g = \frac{10}{180} \times \frac{36}{180} \times \frac{36}{180} = 2.22e^{-03}$$

$$hu + g = \frac{10}{180} \times \frac{36}{180} = 1.11e^{-02}$$

$$h + ug = \frac{10}{180} \times \frac{36}{180} = 1.11e^{-02}$$

hug = 0

Tokenization: *subword-level*

Unigram language modeling tokenization (Kudo, 2018)

Corpus

Iteration 1
Step 1

token

hug

$$hu + g = \frac{10}{180} \times \frac{36}{180} = 1.11e^{-02}$$

pug

$$pu + g = \frac{10}{180} \times \frac{36}{180} = 1.33e^{-02}$$

lug

$$lu + g = \frac{10}{180} \times \frac{36}{180} = 5.56e^{-03}$$

bug

$$bu + g = \frac{10}{180} \times \frac{36}{180} = 4.44e^{-03}$$

dug

$$du + g = \frac{10}{180} \times \frac{36}{180} = 5.56e^{-03}$$

$$\mathcal{L} = \sum freq \times (-\log P(word))$$

$$\mathcal{L} = 170.40$$

Tokenization: *subword-level*

Unigram language modeling tokenization (Kudo, 2018)

Corpus

token
hug
pug
lug
bug
dug

Iteration 1

Step 2

Vocabulary

h	hu
u	ug
g	pu
l	lu
b	du
d	bu
p	

$$\mathcal{L} = 170.40$$

hug

$$h + u + g = \frac{10}{180} \times \frac{36}{180} \times \frac{36}{180} = 2.22e^{-03}$$

$$hu + g = \frac{10}{180} \times \frac{36}{180} = 1.11e^{-02}$$

$$h + ug = \frac{10}{180} \times \frac{36}{180} = 1.11e^{-02}$$

$$hug = 0$$

Tokenization: *subword-level*

Unigram language modeling tokenization (Kudo, 2018)

Corpus

token
hug
pug
lug
bug
dug

Iteration 1

Step 2

Vocabulary

h	hu	$\mathcal{L} = 170.40$
u	ug	$\mathcal{L} = 170.40$
g	pu	$\mathcal{L} = 170.40$
l	lu	$\mathcal{L} = 170.40$
b	du	$\mathcal{L} = 170.40$
d	bu	$\mathcal{L} = 170.40$
p		

Tokenization: *subword-level*

Unigram language modeling tokenization (Kudo, 2018)

Corpus

token
hug
pug
lug
bug
dug

Iteration 1

Step 2

Vocabulary

h	hu	$\mathcal{L} = 170.40$
u	ug	$\mathcal{L} = 170.40$
g	pu	$\mathcal{L} = 170.40$
l	lu	$\mathcal{L} = 170.40$
b	du	$\mathcal{L} = 170.40$
d	bu	$\mathcal{L} = 170.40$
p		

Tokenization: *subword-level*

Unigram language modeling tokenization (Kudo, 2018)

Corpus

token

hug

pug

lug

bug

dug

Iteration 2

Step 1

hug

$$h + u + g = \frac{10}{144} \times \frac{36}{144} \times \frac{36}{144} = 4.34e^{-03}$$

$$\text{hu} + g = \frac{10}{144} \times \frac{36}{144} = 1.11e^{-02}$$

$$h + ug = 0$$

$$\text{hug} = 0$$

Tokenization: *subword-level*

Unigram language modeling tokenization (Kudo, 2018)

Corpus

Iteration 2
Step 1

$$\mathcal{L} = 168.20$$

token

hug

$$h + u + g = \frac{10}{144} \times \frac{10}{144} \times \frac{36}{144} = 4.34e^{-03}$$

pug

$$pu + g = 2.08e^{-02}$$

lug

$$lu + g = 8.68e^{-03}$$

bug

$$bu + g = 6.94e^{-03}$$

dug

$$du + g = 8.68e^{-03}$$

Tokenization: *subword-level*

Unigram language modeling tokenization (Kudo, 2018)

Corpus

token

hug

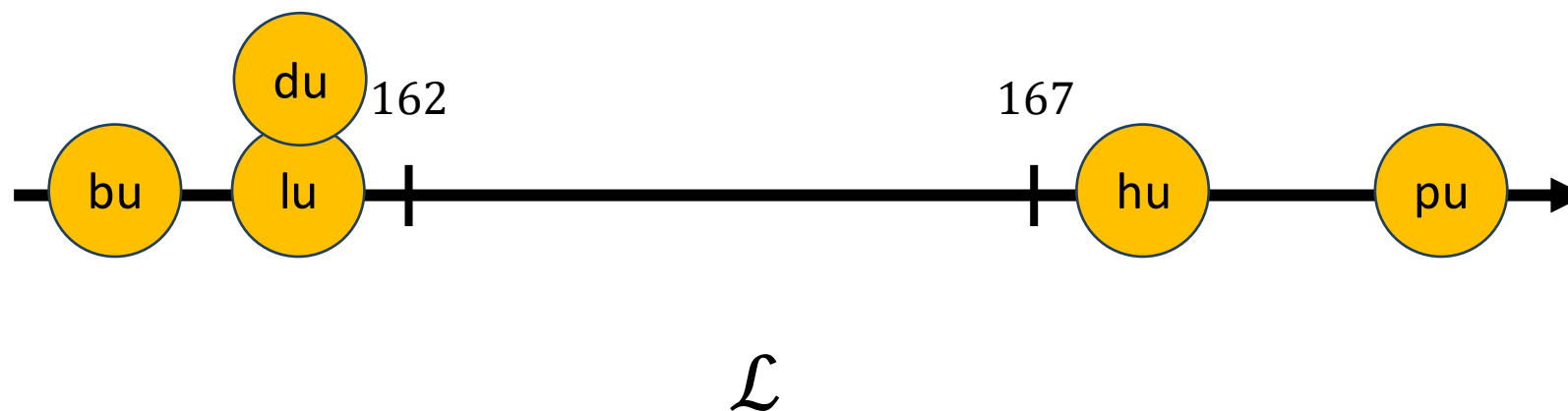
pug

lug

bug

dug

Iteration 2
Step 2



Tokenization: *subword-level*

Unigram language modeling tokenization (Kudo, 2018)

Corpus

token

hug

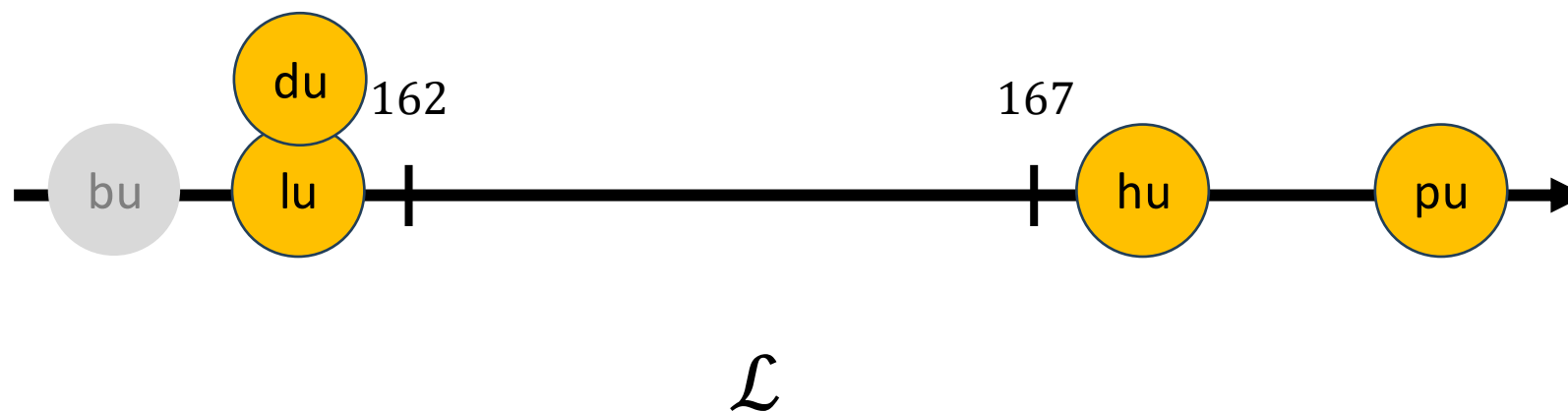
pug

lug

bug

dug

Iteration 2
Step 2



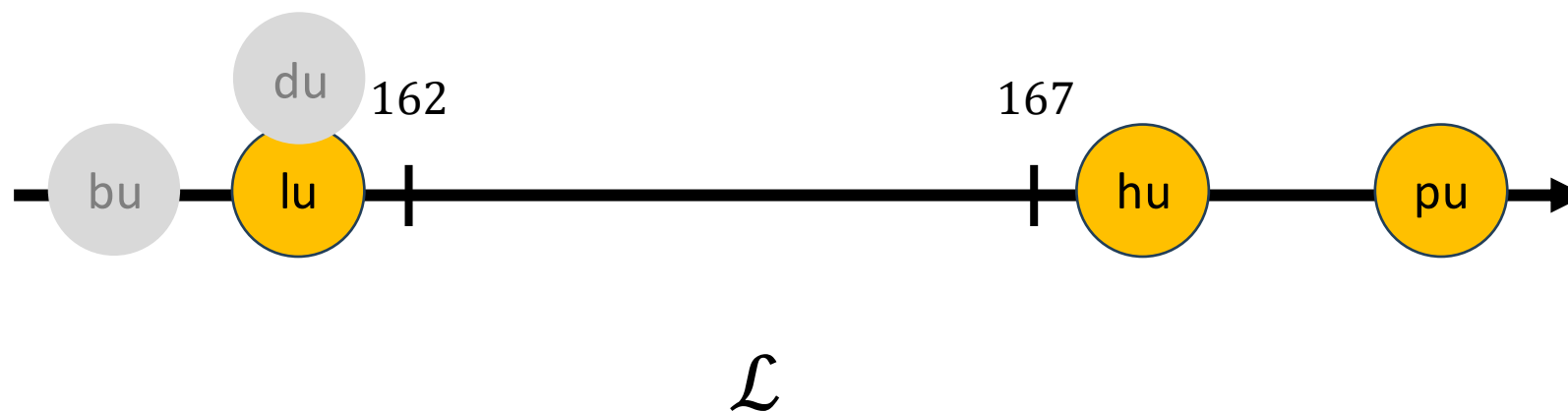
Tokenization: *subword-level*

Unigram language modeling tokenization (Kudo, 2018)

Corpus

token
hug
pug
lug
bug
dug

Iteration 2
Step 2



Tokenization: *subword-level*

BPE vs. ULM

```
my_tokenizers = [  
    ('BPE 8k', load_tokenizer('bpe', 8)),  
    ('BPE 32k', load_tokenizer('bpe', 32)),  
    ('Unigram LM 8k', load_tokenizer('ulm', 8)),  
    ('Unigram LM 32k', load_tokenizer('ulm', 32)),  
]  
FormattedTable(my_tokenizers).print_table_from_words("stabilizing",  
"destabilizing")
```

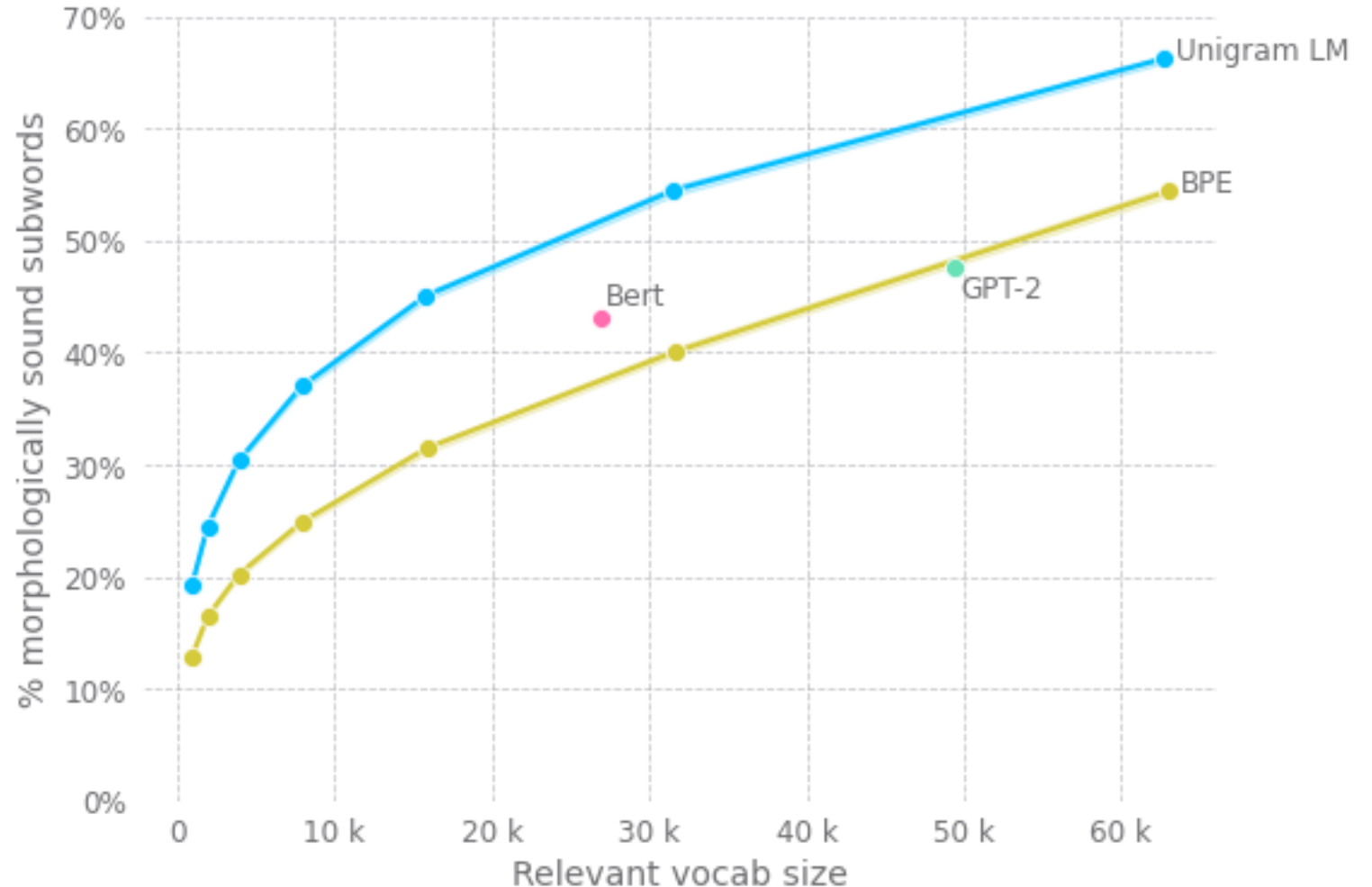
	stabilizing	destabilizing
BPE 8k	st-ab-il-izing	dest-ab-il-izing
BPE 32k	stabil-izing	dest-abil-izing
Unigram LM 8k	sta-bil-izing	de-sta-bil-izing
Unigram LM 32k	stabiliz-ing	de-stabiliz-ing

NOTE: unigram LMs recover common suffixes like 'ly', 's' and 'ing' much more often than BPE.

Tokenization: *subword-level*

BPE vs. ULM

Unigram LM method tends to produce more tokens that are morphologically interpretable

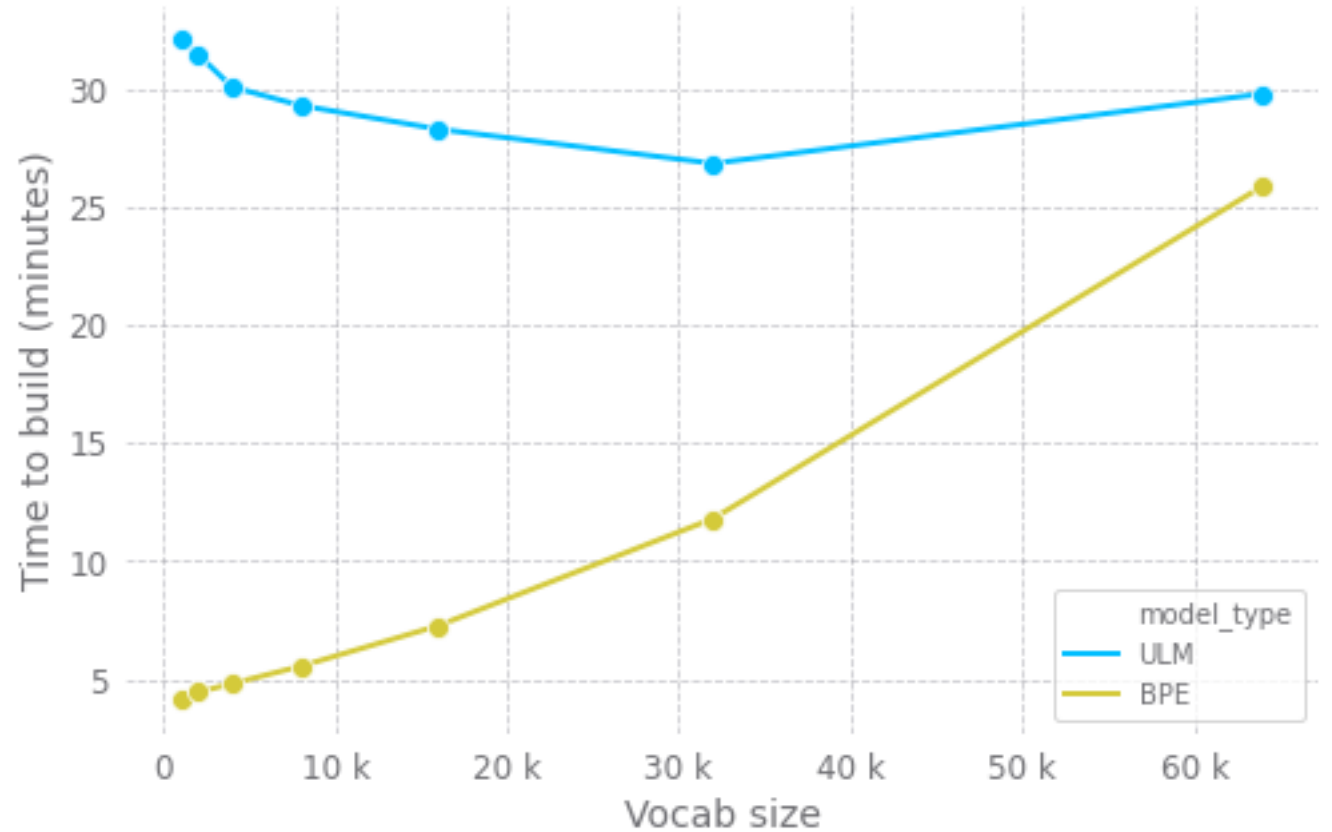


Tokenization: *subword-level*

BPE vs. ULM

SPEED:

Unigram LM gets faster to learn as the vocab size increases.

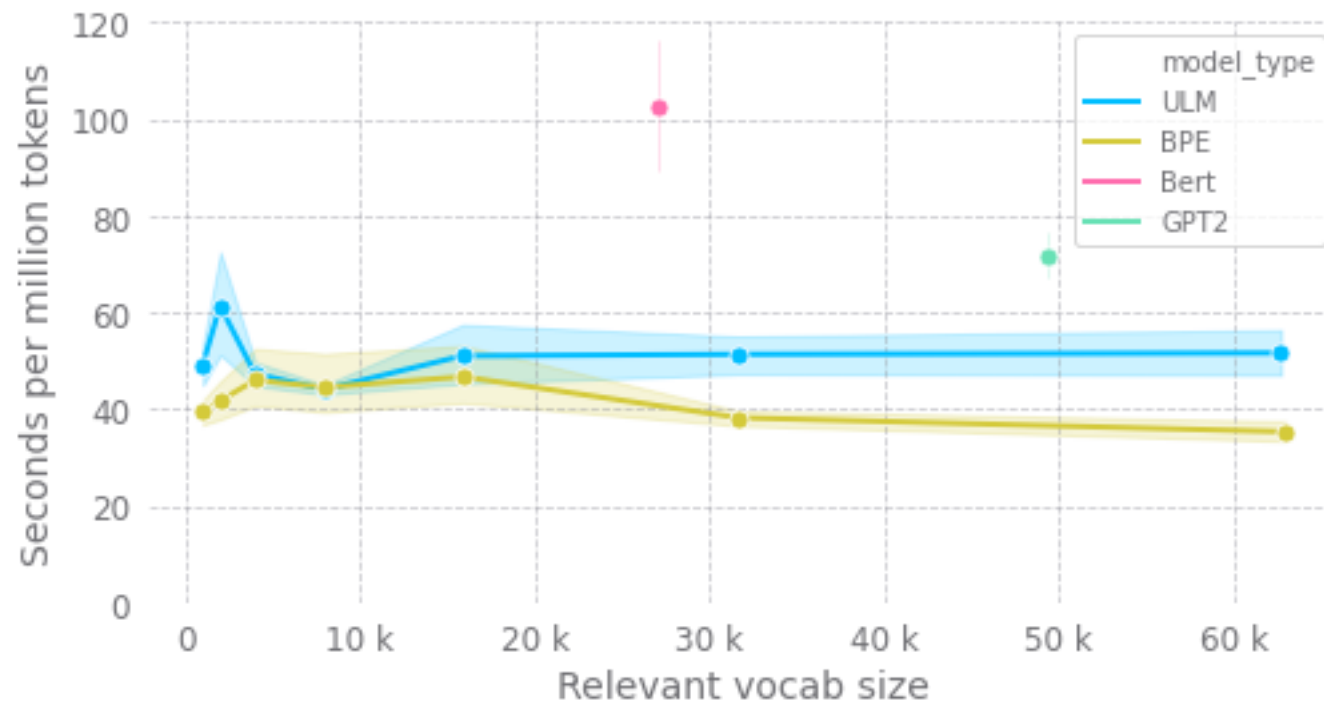


Tokenization: *subword-level*

BPE vs. ULM

INFERENCE:

Difference is insignificant



Tokenization: *subword-level*

Common algorithms:

- Byte-Pair Encoding (BPE) (Sennrich et al., 2016)
- Unigram language modeling tokenization (Kudo, 2018)
- **WordPiece (Schuster and Nakajima, 2012)**
- SentencePiece (Kudo and Richardson, 2018)

Tokenization: *subword-level*

WordPiece (Schuster and Nakajima, 2012)

- Initially developed by Google to pretrain BERT
- Reused in quite a few Transformer models based on BERT, such as DistilBERT, MobileBERT, Funnel Transformers, and MPNET.
- Very similar to BPE in terms of the training, but the actual tokenization is done differently.

Tokenization: *subword-level*

WordPiece (Schuster and Nakajima, 2012)

METHOD:

- Identify subwords by adding a prefix (like ## for BERT), each word is initially split by adding that prefix to all the characters inside the word.
 - Example: 'word' → w ##o ##r ##d
- Like BPE, learns merge rules. The main difference is the way the pair to be merged is selected: Instead of selecting the most frequent pair, WordPiece computes a score for each pair, using the following formula:

$$score = \frac{freq_of_pair}{(freq_of_first_element \times freq_of_second_element)}$$

Highest score → MERGE

- Repeat merging till desired vocabulary size is achieved.

Tokenization: *subword-level*

WordPiece (Schuster and Nakajima, 2012)

IF `freq_of_pair` is the same for two words

BUT

`[freq_of_first_element_w1, freq_of_second_element_w1]` → small

`[freq_of_first_element_w2, freq_of_second_element_w2]` → large

THEN

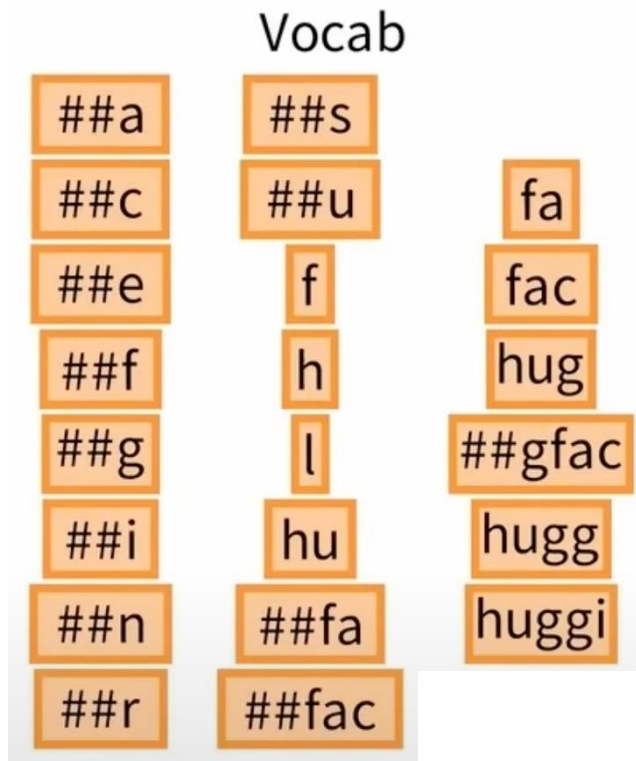
`Score_w1 > Score_w2` →

Prioritize the merging of pairs where the individual parts are less frequent in the vocabulary

Tokenization: *subword-level*

WordPiece (Schuster and Nakajima, 2012)

At the time tokenization

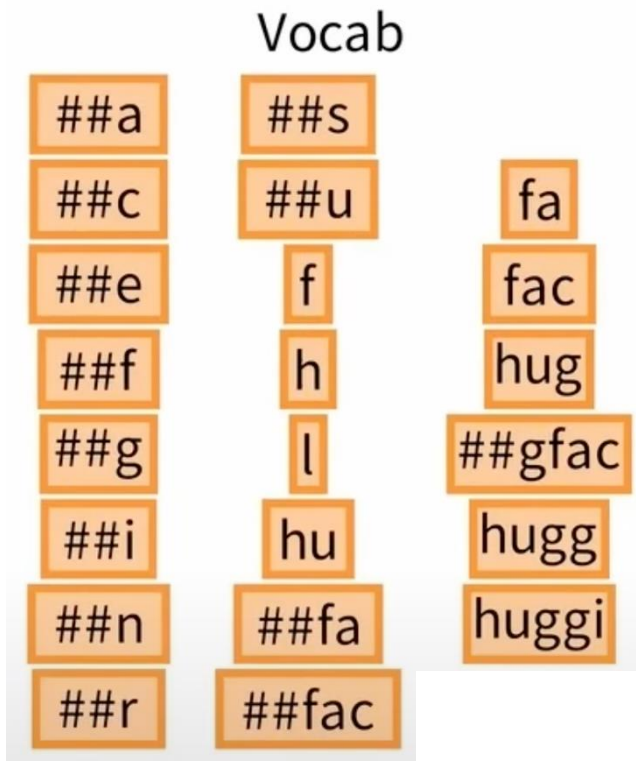


huggingface

Tokenization: *subword-level*

WordPiece (Schuster and Nakajima, 2012)

At the time tokenization

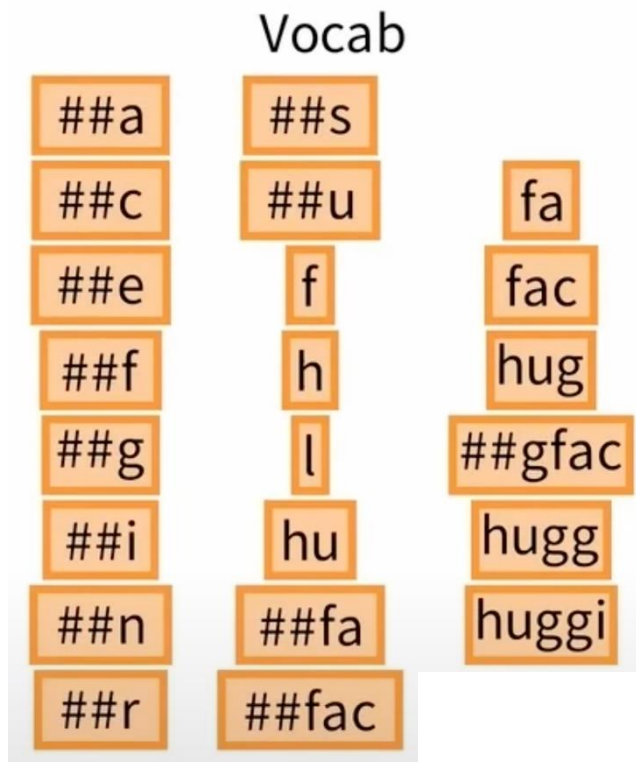


h u g g i n g f a c e

Tokenization: *subword-level*

WordPiece (Schuster and Nakajima, 2012)

At the time tokenization

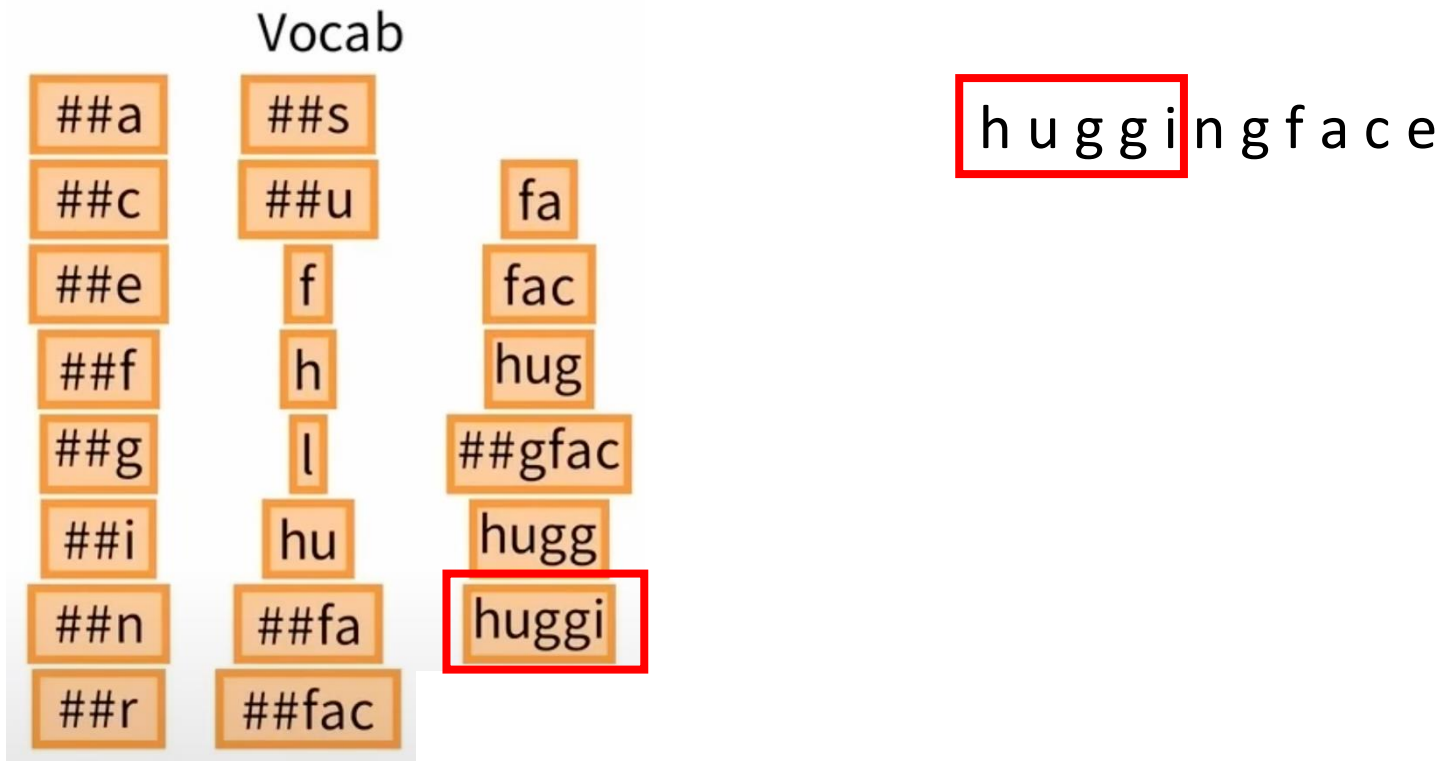


h u g g i n g f a c e

Tokenization: *subword-level*

WordPiece (Schuster and Nakajima, 2012)

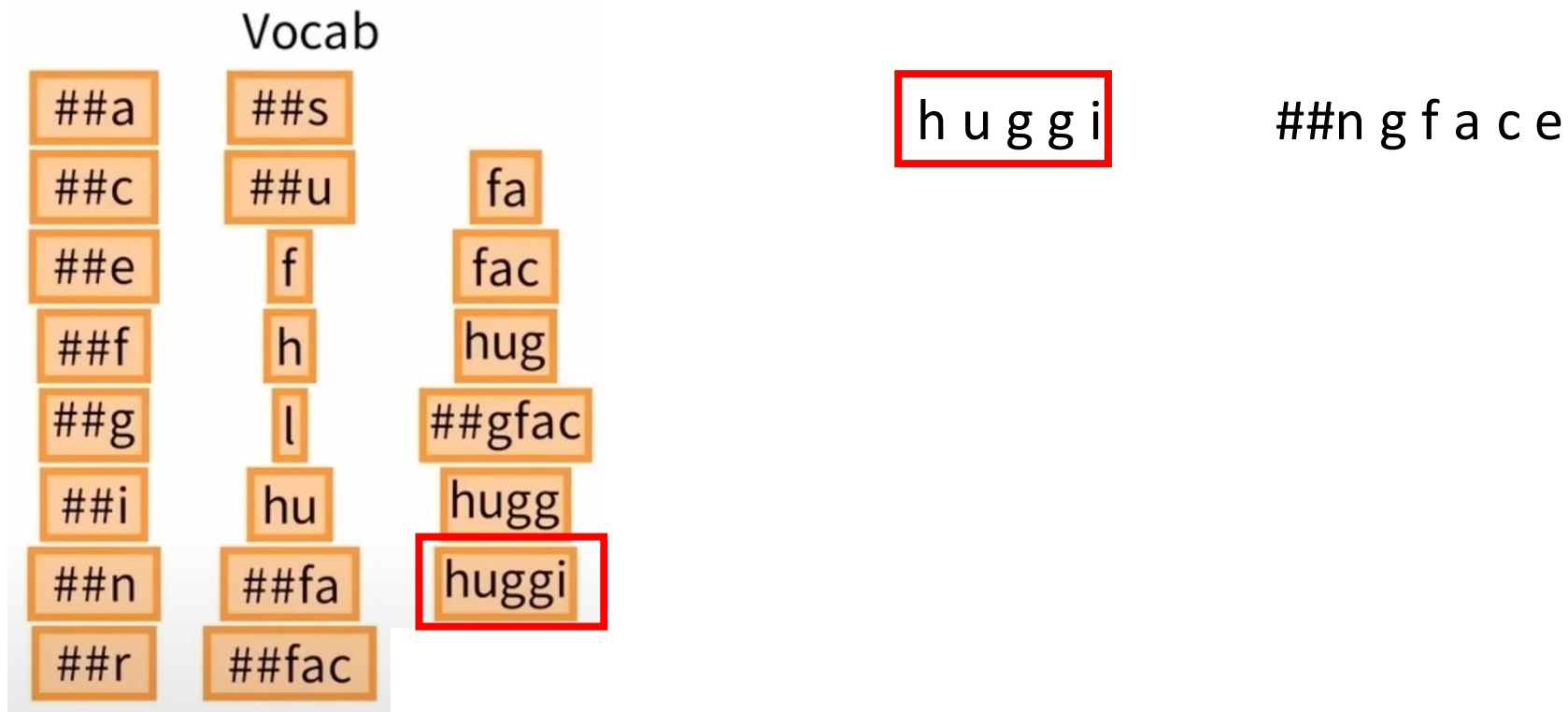
At the time tokenization



Tokenization: *subword-level*

WordPiece (Schuster and Nakajima, 2012)

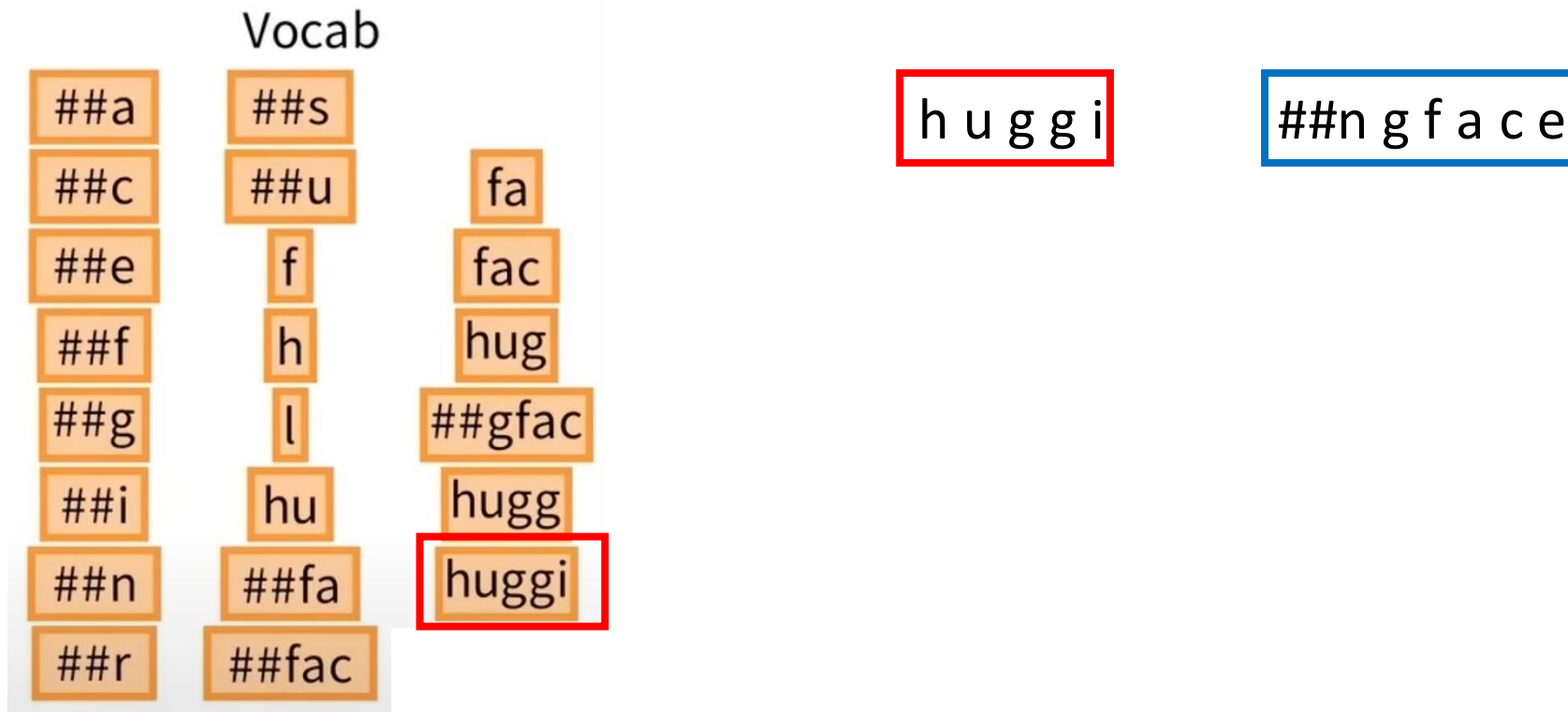
At the time tokenization



Tokenization: *subword-level*

WordPiece (Schuster and Nakajima, 2012)

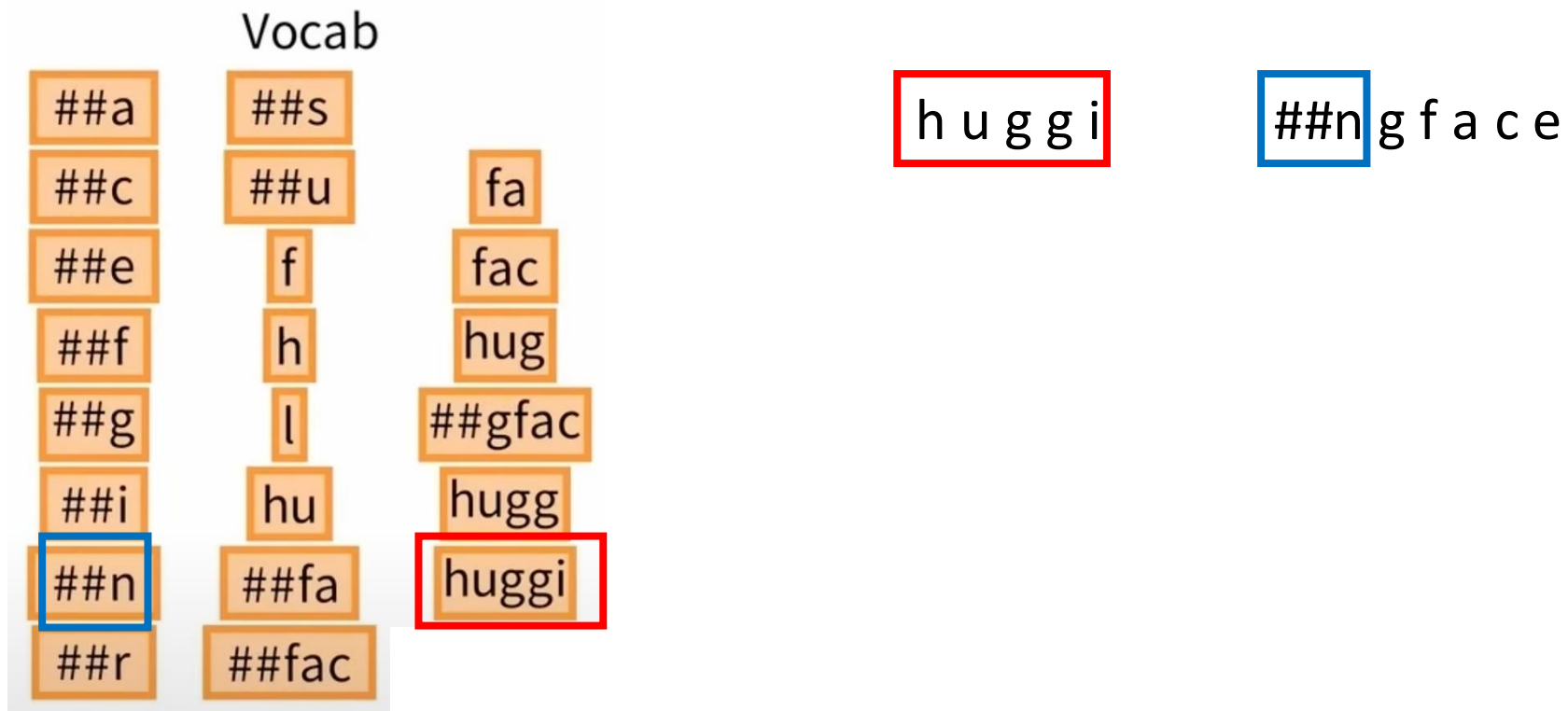
At the time tokenization



Tokenization: *subword-level*

WordPiece (Schuster and Nakajima, 2012)

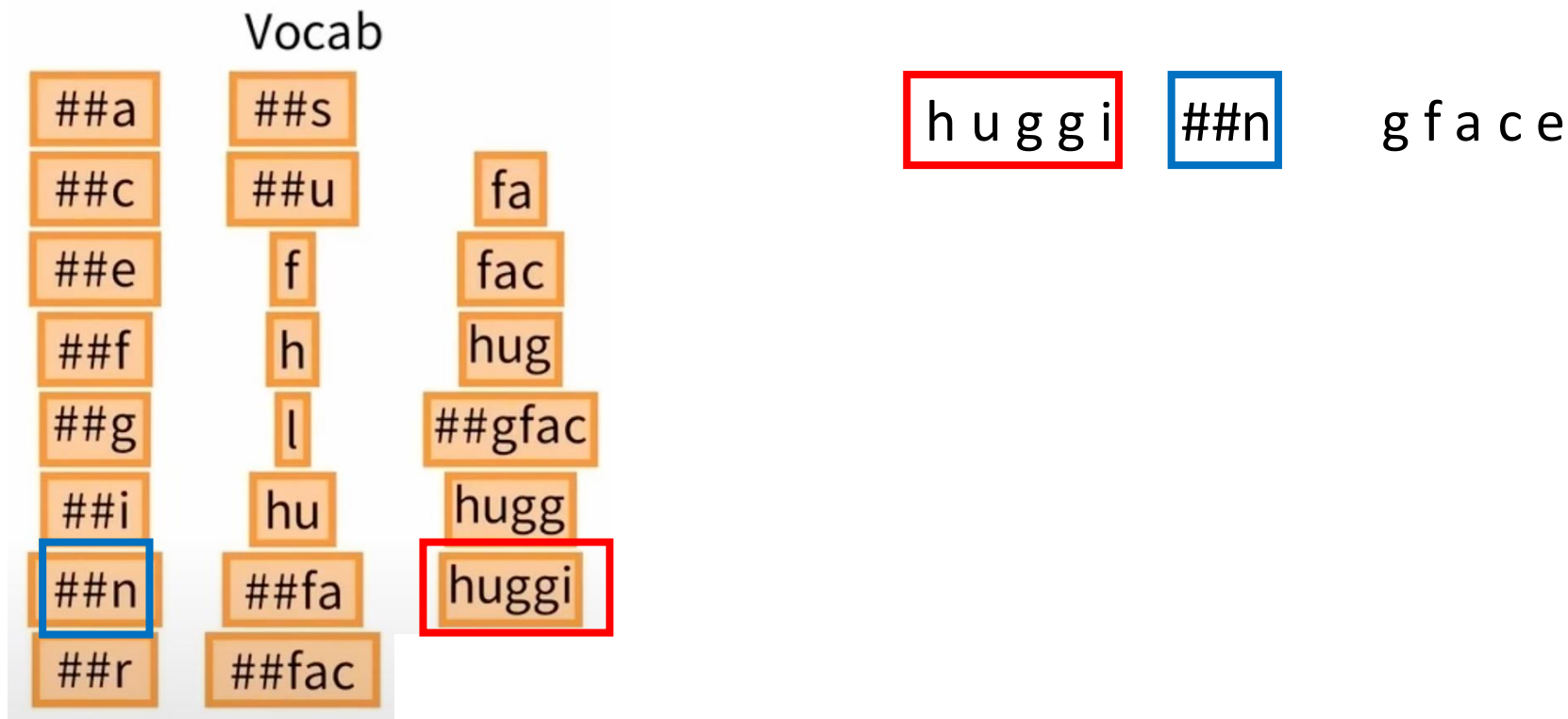
At the time tokenization



Tokenization: *subword-level*

WordPiece (Schuster and Nakajima, 2012)

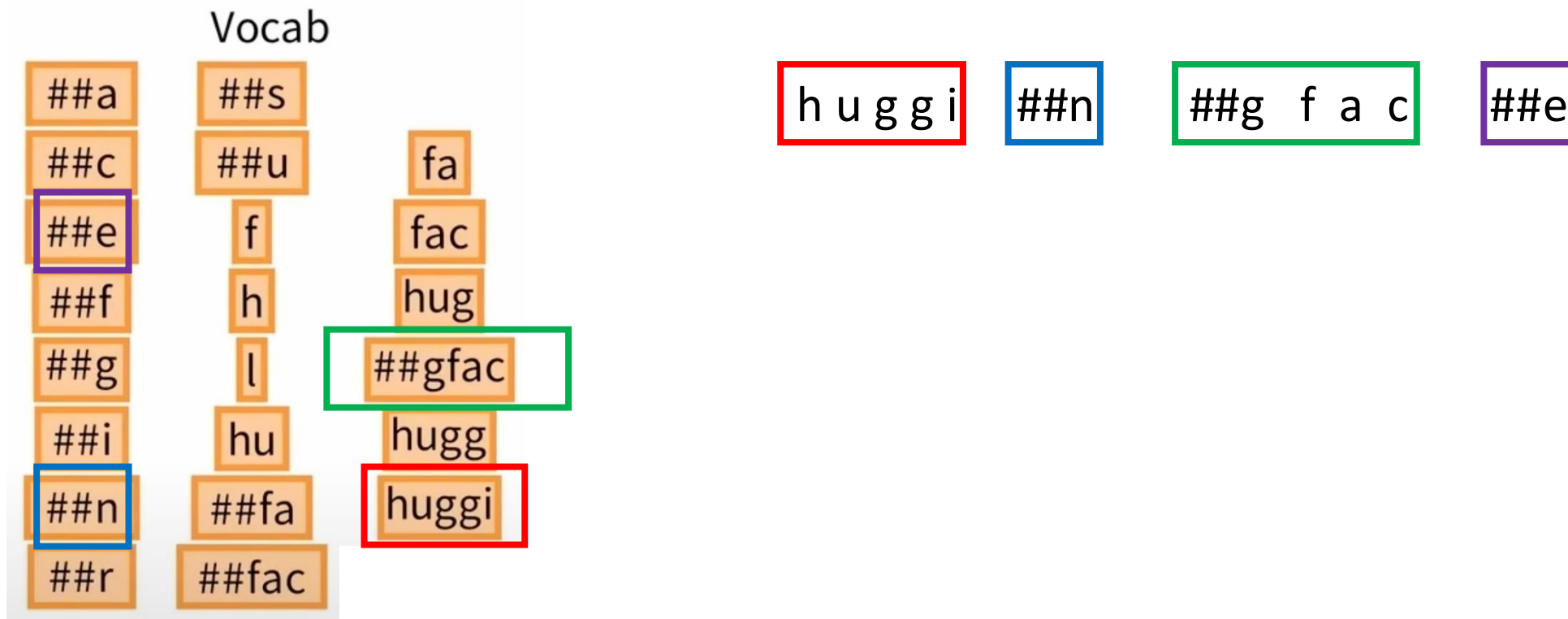
At the time tokenization



Tokenization: *subword-level*

WordPiece (Schuster and Nakajima, 2012)

At the time tokenization



Tokenization: *subword-level*

Common algorithms:

- Byte-Pair Encoding (BPE) (Sennrich et al., 2016)
- Unigram language modeling tokenization (Kudo, 2018)
- WordPiece (Schuster and Nakajima, 2012)
- **SentencePiece (Kudo and Richardson, 2018)**

Tokenization: *subword-level*

SentencePiece (Kudo and Richardson, 2018)

- Proposed as a **language-independent** and **pre-tokenization-free** tokenizer that can directly model raw text sequences.
- Enables uniform preprocessing and integrates seamlessly into modern neural pipelines.

Tokenization: *subword-level*

SentencePiece (Kudo and Richardson, 2018)

- SentencePiece is used to tokenize input for XLNet ([Yang et al. 2019](#)) and mBART ([Liu et al. 2020](#)).

Tokenization: *subword-level*

SentencePiece (Kudo and Richardson, 2018)

- 1. Treat the input as a sequence of Unicode characters:**
 - operates on raw text, not on whitespace-separated words.
 - encodes even the space character (as a special symbol `_`)
 - Example:
"Hello world" becomes ["Hello", "_world"]
- 2. Unsupervised subword training using either:**
 - BPE (Byte-Pair Encoding)
 - Unigram Language Model

Tokenization: *subword-level*

Byte-Pair Encoding (BPE)

Core Idea:

Iteratively merge the most frequent pair of characters or character sequences in training data.

Algorithm:

1. Start with a character-level vocabulary.
2. Identify the most frequent character pair in the corpus.
3. Merge this pair into a new subword token.
4. Repeat until a predefined vocabulary size is reached.

Tokenization: *subword-level*

Unigram language modeling tokenization

Core Idea:

Model tokenization as a probabilistic language modeling problem.

Algorithm:

1. Start with a large initial vocabulary of subwords.
2. Use an iterative process to prune subwords based on their likelihood to form a language model.
3. Retain tokens that maximize the probability of the observed data.

Tokenization: *subword-level*

WordPiece

Core Idea:

Similar to BPE, but merges based on maximizing language modeling likelihood, not solely frequency.

Algorithm:

1. Initialize vocabulary with characters.
2. Iteratively merge subwords that increase likelihood of training data under a language model.
3. Use this vocabulary to tokenize text.

Tokenization: *subword-level*

SentencePiece

Core Idea:

Provides a unified approach that directly handles raw text input without requiring pre-tokenization.

Key Features:

- Capable of implementing BPE, Unigram, and other tokenizers within the same framework.
- Avoids assumptions about input text formatting (e.g., spaces, punctuation).

Tokenization: *subword-level*

Comparison

Tokenizer	Basis for Merging	Computation	Commonly Used In
BPE	Frequency-based	Efficient, Greedy	GPT models, OpenAI models
Unigram	Probabilistic	More expensive	T5, Japanese tokenization
WordPiece	Likelihood-based	Moderately efficient	BERT, DistilBERT
SentencePiece	Raw text handling	Flexible	Multilingual models (e.g., mBERT, XLM)

Tokenization: *subword-level*

Compare algorithms:

"This is an example of advanced NLP tokenization."

Byte-Pair Encoding (BPE)

```
["This", "is", "an", "ex", "am", "ple", "of", "ad", "van", "ced", "NLP", "token", "ization", "."]
```

Unigram Language Model Tokenization

```
["This", "is", "an", "example", "of", "advanced", "N", "LP", "token", "ization"]
```

WordPiece

```
["This", "is", "an", "ex", "amp", "le", "of", "advan", "ced", "NLP", "token", "ization"]
```

SetencePiece-BPE

```
["_This", "_is", "_an", "_ex", "am", "ple", "_of", "_advan", "ced", "_NLP", "_token", "ization", "."]
```

SentencePiece-Unigram

```
["_This", "_is", "_an", "_example", "_of", "_advanced", "_NLP", "_token", "ization", "."]
```

Tokenization: *challenges*

Ambiguity: Tokenization can be ambiguous, especially in languages where words can have multiple meanings. For example, the word “run” could refer to a physical activity, a computer program, or the act of managing or operating something. This ambiguity can make it difficult for a tokenizer to accurately break down the text into individual tokens.

Tokenization: *challenges*

Ambiguity: Tokenization can be ambiguous, especially in languages where words can have multiple meanings. For example, the word “run” could refer to a physical activity, a computer program, or the act of managing or operating something. This ambiguity can make it difficult for a tokenizer to accurately break down the text into individual tokens.

Out-of-vocabulary (OOV) words: Tokenization can also encounter OOV words, which are words that are not in the tokenizer’s vocabulary. OOV words can be problematic, especially in domain-specific texts, where they occur frequently.

Tokenization: *challenges*

Ambiguity: Tokenization can be ambiguous, especially in languages where words can have multiple meanings. For example, the word “run” could refer to a physical activity, a computer program, or the act of managing or operating something. This ambiguity can make it difficult for a tokenizer to accurately break down the text into individual tokens.

Out-of-vocabulary (OOV) words: Tokenization can also encounter OOV words, which are words that are not in the tokenizer’s vocabulary. OOV words can be problematic, especially in domain-specific texts, where they occur frequently.

Noise: Large text documents can contain a lot of noise, including typos, abbreviations, and special characters that can hinder tokenization.

Tokenization: *challenges*

Ambiguity: Tokenization can be ambiguous, especially in languages where words can have multiple meanings. For example, the word “run” could refer to a physical activity, a computer program, or the act of managing or operating something. This ambiguity can make it difficult for a tokenizer to accurately break down the text into individual tokens.

Out-of-vocabulary (OOV) words: Tokenization can also encounter OOV words, which are words that are not in the tokenizer’s vocabulary. OOV words can be problematic, especially in domain-specific texts, where they occur frequently.

Noise: Large text documents can contain a lot of noise, including typos, abbreviations, and special characters that can hinder tokenization.

Language-specific challenges: Different languages have unique challenges when it comes to tokenization. For example, languages with complex writing systems, such as Chinese and Japanese, require specialized tokenization methods.

Tokenization: *in Python*

Built-in: `split()`

Syntax

```
string.split(separator, maxsplit)
```

Parameter Values

Parameter	Description
<i>separator</i>	Optional. Specifies the separator to use when splitting the string. By default any whitespace is a separator
<i>maxsplit</i>	Optional. Specifies how many splits to do. Default value is -1, which is "all occurrences"

By default, `split()` use whitespace as separator, but we can change it to anything.

```
text = """There are multiple ways we can perform tokenization on given text data.""" # Split text by whitespace
tokens = text.split()
print(tokens)
```

```
> ['There', 'are', 'multiple', 'ways', 'we', 'can', 'perform', 'tokenization', 'on', 'given', 'text', 'data.']
```

Tokenization: *in Python*

Regular Expression

Function	Description
<u>findall</u>	Returns a list containing all matches
<u>search</u>	Returns a <u>Match object</u> if there is a match anywhere in the string
<u>split</u>	Returns a list where the string has been split at each match
<u>sub</u>	Replaces one or many matches with a string

```
import re
text = """There are multiple ways we can perform tokenization on given text data."""
tokens = re.findall("[\w]+", text) print(tokens)
```

```
> ['There', 'are', 'multiple', 'ways', 'we', 'can', 'perform', 'tokenization', 'on', 'given', 'text', 'data']
```


Tokenization: *in Python*

Natural Language Toolkit (NLTK) Library

```
!pip install --user -U nltk

from nltk.tokenize import word_tokenize

text = """There are multiple ways we can perform tokenization on given text data."""
tokens = word_tokenize(text)

> ['There', 'are', 'multiple', 'ways', 'we', 'can', 'perform', 'tokenization', 'on', 'given', 'text', 'data', '.']
```

NOTE: NLTK word tokenization also consider the punctuation as token.

Tokenization: *in Python*

spaCy Library

```
!pip install spacy
!python -m spacy download en
```

```
# Load English model from spacy
from spacy.lang.en import English
```

```
# Load English tokenizer.
# nlp object will be used to create 'doc' object which uses preprecoessing pipeline's components such as tagger,
parser, NER and word vectors
nlp = English()
```

```
text = """There are multiple ways we can perform tokenization on given text data."""
```

```
# Now we will process above text using 'nlp' object. Which is use to create documents with linguistic annotations
and various nlp properties
my_doc = nlp(text)
```

```
# Above step has already tokenized our text but its in doc format, so lets write for loop to create list of it
token_list = []
for token in my_doc:
    token_list.append(token.text)

print(token_list)
```

```
> ['There', 'are', 'multiple', 'ways', 'we', 'can', 'perform', 'tokenization', 'on', 'given', 'text', 'data', '.']
```

Tokenization: *in Python*

keras Library

By default, this function automatically does 3 things:

- Splits words by space (split=" ").
- Filters out punctuation (filters='!"#\$%&()*+,-./:;<=>?@[\\]^_`{|}~\t\n').
- Converts text to lowercase (lower=True).

```
!pip install keras
```

```
from keras.preprocessing.text import text_to_word_sequence
```

```
text = """There are multiple ways we can perform tokenization on given text data."""
```

```
tokens = text_to_word_sequence(text)  
print(tokens)
```

```
> ['there', 'are', 'multiple', 'ways', 'we', 'can', 'perform', 'tokenization', 'on', 'given', 'text', 'data']
```

NOTE: all words are also converted to lowercase.

Tokenization: *in Python*

gensim Library

```
!pip install gensim

from gensim.utils import tokenize

text = """There are multiple ways we can perform tokenization on given text data."""

tokens = list(tokenize(text))
print(tokens)

> ['There', 'are', 'multiple', 'ways', 'we', 'can', 'perform', 'tokenization', 'on', 'given', 'text', 'data']
```

Tokenization: References

Implementation

BPE: <https://huggingface.co/learn/nlp-course/chapter6/5?fw=pt>

WordPiece: <https://huggingface.co/learn/nlp-course/chapter6/6?fw=pt>

Unigram: <https://huggingface.co/learn/nlp-course/chapter6/7?fw=pt>

Papers

- Byte-Pair Encoding (BPE) ([Sennrich et al., 2016](#))
- Unigram language modeling tokenization ([Kudo, 2018](#))
- WordPiece ([Schuster and Nakajima, 2012](#))
- SentencePiece ([Kudo and Richardson, 2018](#))

Text Processing Fundamentals

- Regular Expressions
- Tokenization
- **Normalization**
- Libraries and Toolkits

Normalization

- A series of steps aimed at converting different linguistic forms of text into a uniform format.
- Crucial for ensuring that the algorithm interprets words or phrases consistently, which is essential for many NLP tasks:
e.g., text classification, information retrieval, and machine translation.

Normalization: *Why?*

- **Feature Reduction:** Normalization reduces the feature space by ensuring that different variants of the same word are treated equally.
- **Improving Matching:** It helps in matching words in search queries and information retrieval systems.
- **Consistency:** For algorithms to perform accurately, they need a consistent format of input data.
- **Performance:** By reducing the number of unique tokens, normalization can improve the computational efficiency of NLP systems.

Normalization

Typical Steps:

- Tokenization
- Removing stop words
- Handling punctuations and special characters
- Handling whitespace, tab, newline
- Case folding
- Expanding contractions (don't -> do not)
- Handling unicode characters - accented letters and some punctuation
- Number words -> numeric
- Stemming
- Lemmatization
- Foreign words
- Handling abbreviations
- [Ontology mapping]

Normalization

Typical Steps:

- Tokenization
- Removing stop words
- Handling punctuations and special characters
- Handling whitespace, tab, newline
- Case folding
- Expanding contractions (don't -> do not)
- Handling unicode characters - accented letters and some punctuation
- Number words -> numeric
- Stemming
- Lemmatization
- Foreign words
- Handling abbreviations
- [Ontology mapping]

Stemming

Process of reducing words to their word stem, base or root form—generally a written word form.

i.e. chop off the end of the word

Julie Beth Lovins wrote the first published stemmer in 1968

<https://aclanthology.org/www.mt-archive.info/MT-1968-Lovins.pdf>

Stemming: *Techniques*

Rule-based

- Lovins
- Porters
- Paice/Husk
- Dawson

Statistical/ML

- N-gram
- HMM¹
- YASS²

Hybrid

1. Hidden Markov Model
2. Yet Another Suffix Stripper

Stemming

Rule-based Algorithm (for English):

Porter's algorithm (*Porter, 1980*): a 60-rule algorithm

STEPS:

1. Separate Words into Consonants and Vowels
2. Measure the Word (m): Words are then measured based on their vowel-consonant patterns.
3. Apply Rules by Stages
 - Stage 1: Handles Plural and Past Participles
 - Stage 2: Handles 'ed', 'ing' Suffixes
 - Stage 3: Handles Words Ending in 'y'
 - Stage 4: Procession of the Suffixes
 - Stage 5: Removal of 'e' or 'l'

Stemming

Other Rule-based Algorithms:

Porter's algorithm (*Porter, 1980*)

Snowball algorithm (also known as the Porter2)

Lancaster algorithm (most aggressive stemming algorithm)

Stemming: *Truncating*

Examples

Word	Porter Stemmer	Snowball Stemmer	Lancaster Stemmer
friend	friend	friend	friend
friendship	friendship	friendship	friend
friends	friend	friend	friend
friendships	friendship	friendship	friend

Stemming: *Statistical/ML*

Statistical stemmers use probabilistic / machine learning techniques (such as decision trees, neural networks) trained on a large corpus of text data to learn patterns and relationships between various forms of a word.

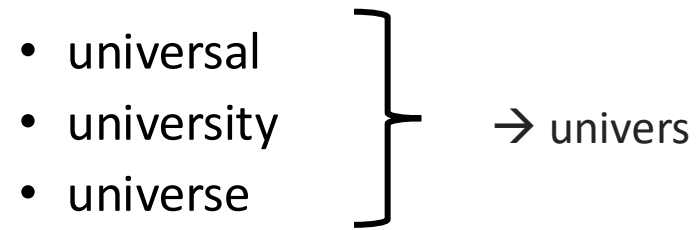
N-gram

- calculates n -grams ($n = 2,3$ usually): breaking words into n length words and applying statistical analysis in order to identify them.
- **Advantage:**
 - Ability to handle irregular word
 - More adaptable to different languages
- **Disadvantage:** requires high memory.

Stemming

Limitations:

- Overstemming: Occurs when two words are stemmed to the same root that are not of the same root.



Stemming

Limitations:

- Overstemming: Occurs when two words are stemmed to the same root that are not of the same root.
- Understemming: Occurs when two words that should be stemmed to the same root are not.
 - alumnus
 - alumni
 - alumnae

Stemming

Limitations:

- Overstemming: Occurs when two words are stemmed to the same root that are not of the same root.
- Understemming: Occurs when two words that should be stemmed to the same root are not.
- Stemming can often create non-words, as it operates on a rule-based method.

Stemming

Limitations:

- Overstemming: Occurs when two words are stemmed to the same root that are not of the same root.
- Understemming: Occurs when two words that should be stemmed to the same root are not.
- Stemming can often create non-words, as it operates on a rule-based method.
- Context is not taken into account
 - e.g., "saw" could be a verb or a noun.

Stemming

Limitations:

- Overstemming: Occurs when two words are stemmed to the same root that are not of the same root.
- Understemming: Occurs when two words that should be stemmed to the same root are not.
- Stemming can often create non-words, as it operates on a rule-based method.
- Context is not taken into account
 - e.g., "saw" could be a verb or a noun.
- Not suitable for all languages

Lemmatization

Process of reducing a word to its base or root form, known as a lemma.

Unlike stemming, which crudely chops off the ends of words, lemmatization involves a linguistic approach that uses vocabulary and detailed morphological analysis of words.

The **lemma** of a word is its base or dictionary form.

Lemmatization

Examples:

```
#> kites ----> kite
#> babies ----> baby
#> dogs ----> dog
#> flying ----> flying
#> smiling ----> smiling
#> driving ----> driving
#> died ----> died
#> tried ----> tried
#> feet ----> foot
```

Stemming and Lemmatization: *in Python*

Libraries:

- 1.WordNet
- 2.TextBlob
- 3.spaCy
- 4.TreeTagger
- 5.Pattern
- 6.Gensim
- 7.Stanford CoreNLP

Lemmatization vs. Stemming

Frequently Asked Questions

Q1. Which is better lemmatization or stemming?

A. The choice depends on the specific use case. Lemmatization produces a linguistically valid word while stemming is faster but may generate non-words.

Q2. Do you do both stemming and lemmatization?

A. Choose based on the task's requirements or context.

Q3. Why is stemming faster than lemmatization?

A. Stemming chops off word endings without considering linguistic context, making it computationally faster. Lemmatization analyzes word forms to determine the base or dictionary form, which takes more processing time.

Lemmatization vs. Stemming

Frequently Asked Questions

Q1. Which is better lemmatization or stemming?

A. The choice depends on the specific use case. Lemmatization produces a linguistically valid word while stemming is faster but may generate non-words.

Q2. Do you do both stemming and lemmatization?

A. Choose based on the task's requirements or context.

Q3. Why is stemming faster than lemmatization?

A. Stemming chops off word endings without considering linguistic context, making it computationally faster. Lemmatization analyzes word forms to determine the base or dictionary form, which takes more processing time.

Lemmatization vs. Stemming

Frequently Asked Questions

Q1. Which is better lemmatization or stemming?

A. The choice depends on the specific use case. Lemmatization produces a linguistically valid word while stemming is faster but may generate non-words.

Q2. Do you do both stemming and lemmatization?

A. Choose based on the task's requirements or context.

Q3. Why is stemming faster than lemmatization?

A. Stemming chops off word endings without considering linguistic context, making it computationally faster. Lemmatization analyzes word forms to determine the base or dictionary form, which takes more processing time.

Text Processing Fundamentals

- Regular Expressions
- Tokenization
- Normalization
- Libraries and Toolkits

Libraries

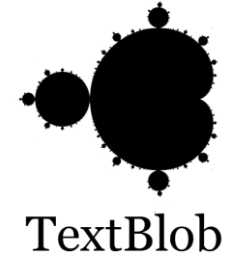
Natural Language Toolkit ([NLTK](#))

- developed by Steven Bird and Edward Loper at the University of Pennsylvania
- **Tools:** classification, stemming, tagging, parsing, semantic reasoning, and tokenization, etc
- Install:
 - `pip install nltk`
 - `conda install -c anaconda nltk`



Libraries

TextBlob



- Built on top of NLTK
- **Tools:** POS Tagging, Noun Phrases, Sentiment Analysis, Tokenization, Lemmatization, etc
- Install:
 - `pip install -U textblob`
 - `python -m textblob.download_corpora`
 - `conda install -c conda-forge textblob`
 - `python -m textblob.download_corpora`

Libraries

Stanza



- A product of Stanford University
- A Python wrapper around CoreNLP (java)
- **Tools:** Grammar tagging, named entity recognition, parsing, sentiment analysis, etc
- Install:
 - `pip install stanza`
 - `conda install -c stanfordnlp stanza`

Libraries

Gensim



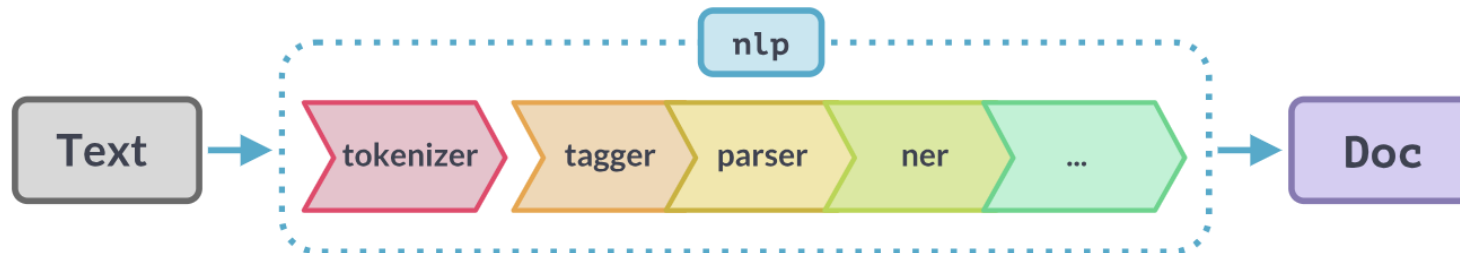
- Developed by Radim Řehůřek piskvorky
- **Tools:** advanced natural language processing: unsupervised topic modeling, document indexing, retrieval by similarity, etc.
- Install:
 - `pip install gensim`
 - `conda install -c conda-forge gensim`

Libraries

spaCy

spaCy

- Developed by Matthew Honnibal and Ines Montani, the founders of the software company Explosion
- **Tools:** advanced natural language processing
- Install:
 - <https://spacy.io/usage>



<https://allenai.github.io/scispacy/>

Libraries

AllenNLP

AllenNLP

- Developed by Allen Institute for AI
- **Tools:** complete platform for solving natural language processing tasks in PyTorch
- Install:
 - `pip install allennlp`
 - `conda install -c conda-forge python=3.8 allennlp`